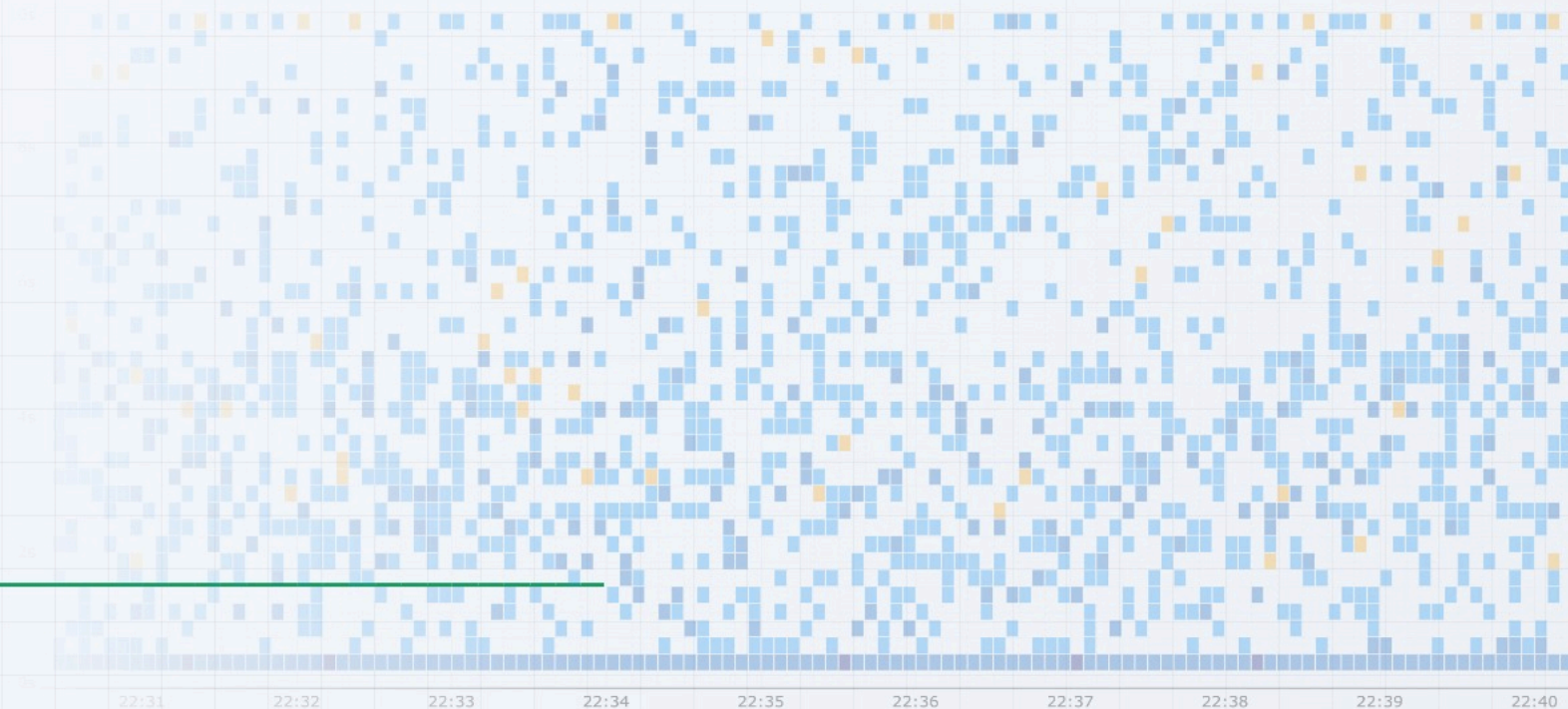


배포 후 1시간, 당신은 무엇을 보고 있나요?

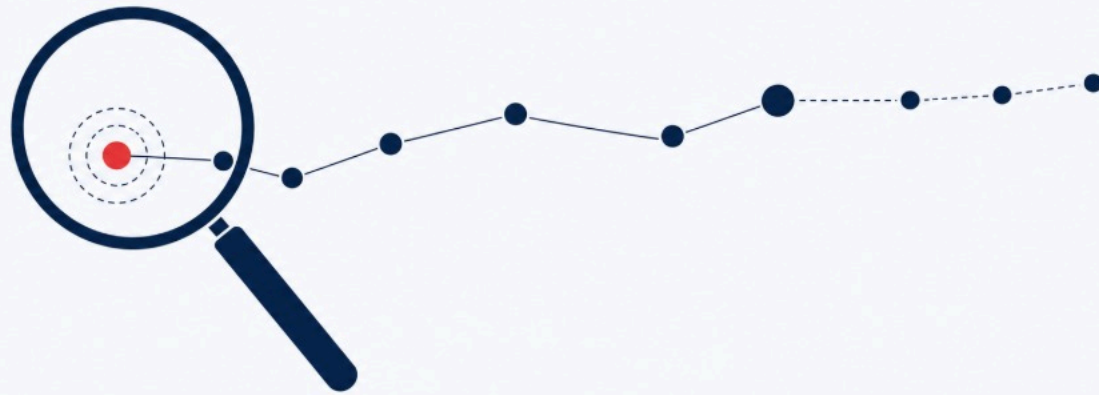
개발의 마지막은 운영이다 — 백엔드 개발자가 문제를 발견하는 법

이헌섭 · WhaTap Labs · 플랫폼팀

KSUG 2026 · Spring User Group



“문제 해결은, 문제를 찾는 것부터.”



“에러가 났는가?” 보다 → “어디서 먼저 시작됐는가?”

“느리다고 합니다” — 정보 없이 받는 세 가지 질문

모니터링이 없으면 — 익숙한 풍경



문제는 “느리다”가 아니라, 어디가 · 언제부터 · 얼마나 모르는 것입니다.

터미널 안에서 헤매다 — 시간 종료

ssh · top · grep · tail — 헤매는 30분

원인 모름 → 시간 지나 상황 종료. 문제가 있다는 건 알지만, 어딘지 모름. 1시간이 지나도 답이 없습니다.

```
$ ssh prod-app-01
Last login: Mon May 18 09:04:12
$ top
PID USER %CPU %MEM TIME+ COMMAND
8821 app 100.0 47.2 4:12.30 java
$ tail -f app.log
...
```

```
$ grep -i ERROR app.log | tail -200
java.lang.NullPointerException
at OrderService.charge(OrderService.java:142)
at ApiController.svc_order(ApiController.java:42)
... 47 more lines
java.lang.NullPointerException
at OrderService.charge(OrderService.java:142)
... 47 more lines
java.lang.OutOfMemoryError: Java heap space
... 200 more lines
```


도구가 아니라, 시각

‘와탭이 얼마나 좋은지’가 아니라 — **모니터링으로 문제를 발견하는 방법과 시각**

증상 발견 → 지표 확인 → 트레이스 · SQL · GC 확인 → 원인 가설

“트래픽 · 성능 · 트러블슈팅” — 같은 단어가 반복됩니다

요즘 채용공고를 보면

트래픽 성능 트러블슈팅

네이버 · 우대사항

대규모 트래픽과 대용량 데이터를 다루는 시스템
효율적 개발·관리

카카오 · 우대사항

성능 측정과 튜닝에 관심. 대용량 트래픽 처리 및
DevOps 경험

라인 · 우대사항

고가용성·대규모 메신저/금융 서비스 운영 경험

토스 · 우대사항

고가용성의 확장 가능한 시스템 설계 운영 + 성능
최적화

우아한형제들 · 우대사항

Application 트러블슈팅 및 성능 최적화 경험

쿠팡 · 자격요건

높은 처리량과 낮은 레이턴시 기반 미션 크리티컬
서비스 구축·관리 경험

서비스가 커질수록, “원인까지 보는 능력”은 우대가 아니라 자격요건이 됩니다.

면접에서도 같은 질문

결국 묻는 것은 — 숫자 · 지표 · 단계

“

어떤 **지표**를 보고, 어떻게 **측정**했고, 숫자가 어떻게 바뀌었는지 말씀해주세요.

“

무엇을 보고 원인을 좁혀가셨는지 — **단계별로** 설명해주세요.

예: p95 ↑ → 트레이스에서 느린 SQL · N+1 발견 → 쿼리 개선 → p95 ↓

문제 해결의 경험은 보는 것부터

볼 수 있었다 — 문제를 찾았다 — 개선했다

볼 수 있었다



문제 발견



성능 개선

측정 못 하면, 답할 수 없다.

업계 블로그도 같은 결론

시작과 끝은 모니터링 — 평소에 읽을 줄 알아야 한밤에 답한다

토스 기술 블로그 · “서버 증설 없이 처리하는 대규모 트래픽” (2024)

“성능 개선의 이터레이션을 진행할 때에도,
시작과 끝은 **모니터링**입니다.

제품이 성장할 때,
서버 개발자는 모니터링 환경을 먼저 구축해야
합니다.

우아한형제들 기술 블로그 · “결제 시스템 성능, 부하, 스트레스 테스트”
(2018)

“이것은 흔히 말하는
**localhost:8080 레벨에서는 확인할 수 없는
문제**였습니다.

제품이 성장할수록 — 평소에 읽던 지표가 배포 직후도, 다음날도, 임계 너머도 답을 만듭니다.

서비스가 성장하면 팀원도 늘고, 배포가 는다

“리팩토링 완료. 아마 빨라졌겠지. PR 머지.” — 이라고 계시진 않나요?



해결된 문제가 다시 발생

응답시간 P95 · 3초 → 0.3초 → 다시 8초

v2.0 · 개선 전

3.0s

v2.1 · 개선 후

0.3s

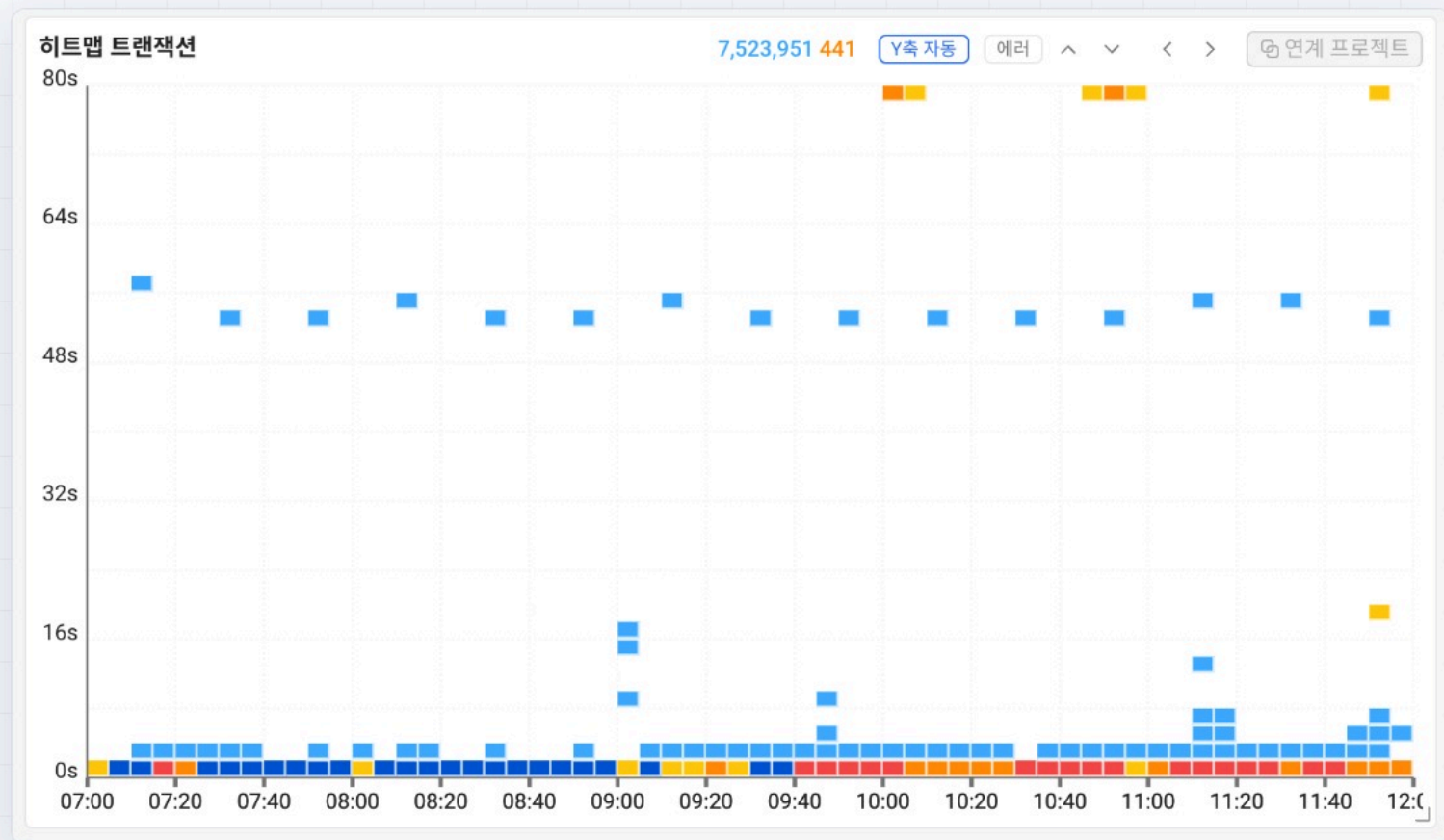
v2.1.1 · 배포 후

8.0s

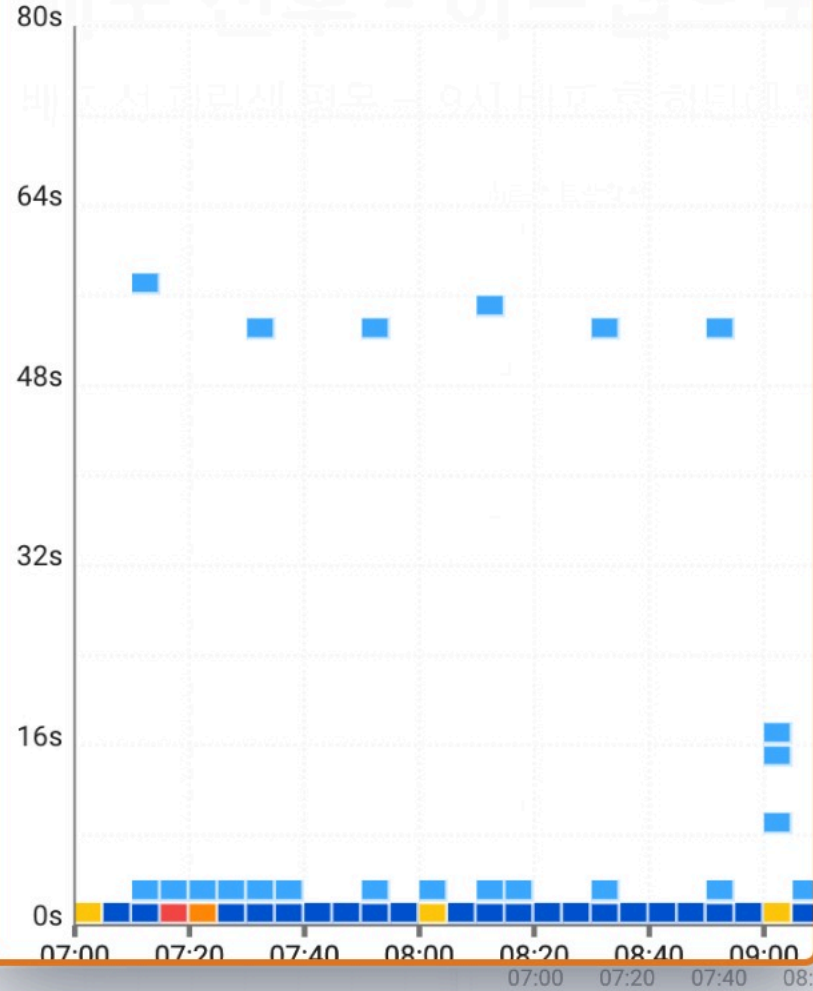
v2.1.1 배포 후 회귀 감지 — 한 번 빨라졌다고 끝난 게 아닙니다.

배포 전후 - 히트맵으로 비교

배포 전 파란색 평온 → 9시 배포 후 하단에 빨간 점(에러)



히트맵 트랜잭션



비교

하단 점(에러)

7,523,951 441

Y축 자동

에러

^ v < >

연계 프로젝트

1 / 3

배포 전 — 전체 파란색, 평온**하단이 파란 점으로 깔림 — 응답 빠르고 에러 없음**

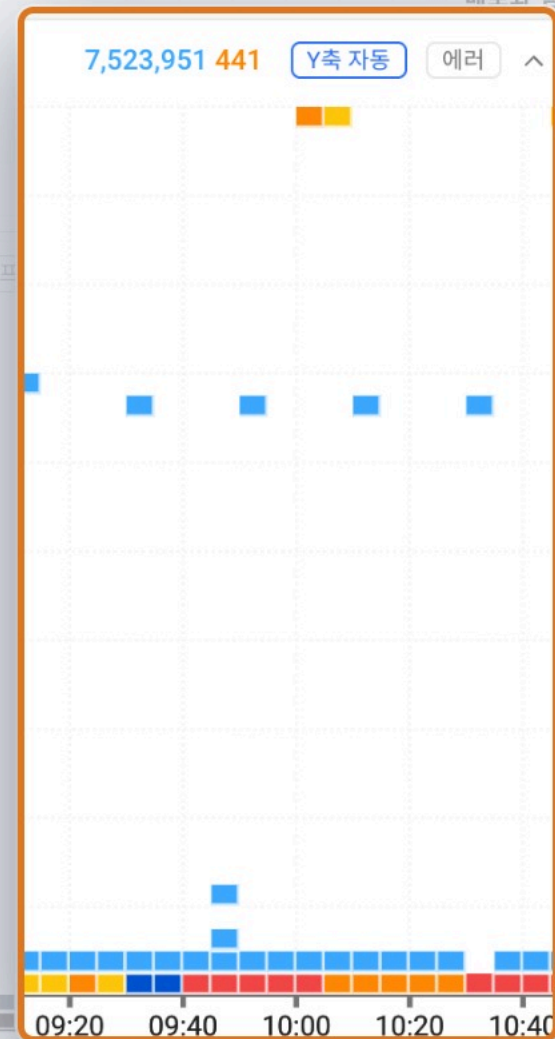
배포 전후 - 히트맵으로 비교

배포 전 파란색 평온 → 9시 배포 후 하단에 빨간 점(에러)

2 / 3

9시 배포 직후 — 빨간 점 출현

응답시간이 높아진 게 아니라, 하단에 에러 (빨강·주황)가 깔린다



배포 전후 - 히트맵으로 비교

3 / 3

배포 전 파란색 평온 → 9시 배포

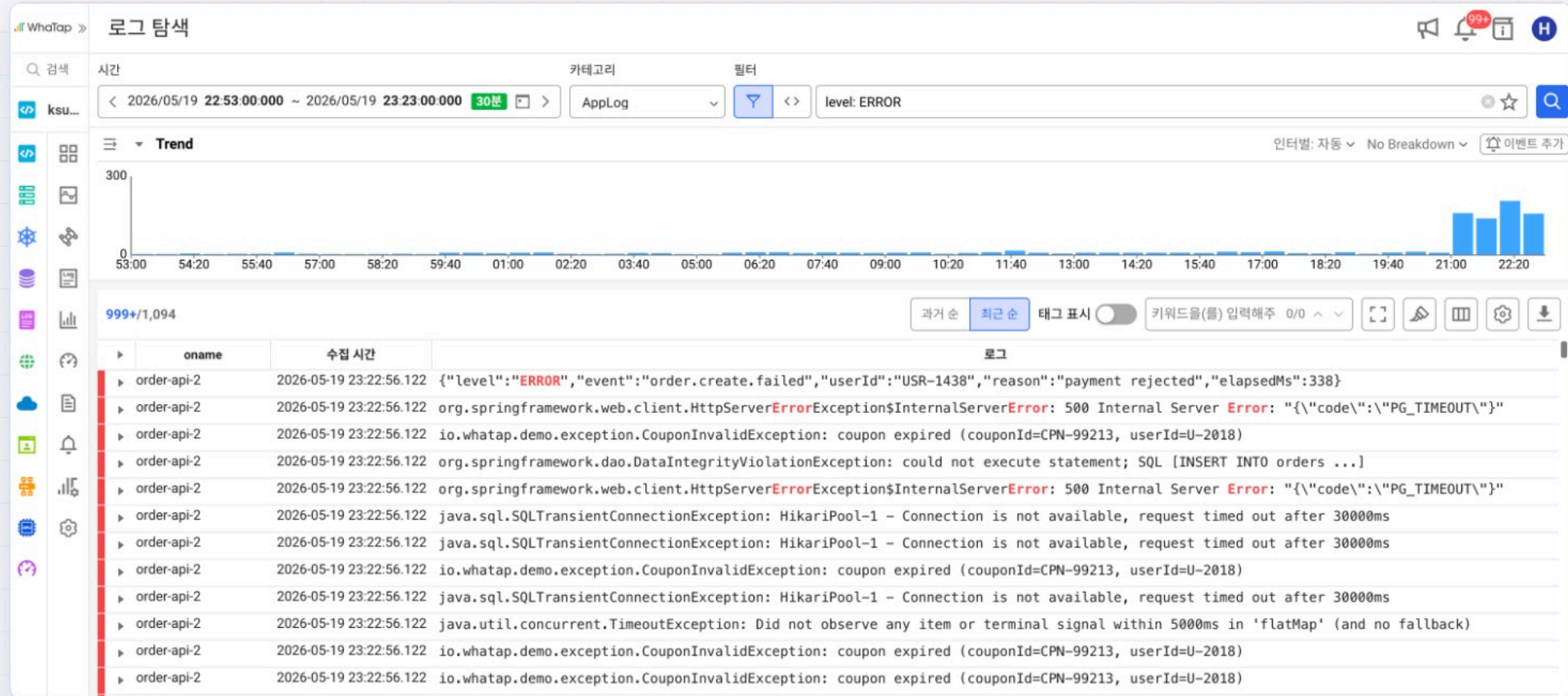
상단 느린 점 — 배포 전부터 계속

위쪽 느린 점들이 배포 전부터 쪽 있었다면, 이번 배포 문제가 아닐 가능성이 높다



배포 한시간 - 로그 패턴을 확인

평시 수십 건 → 짧은 시간에 1,094건. ERROR 로그가 배포 직후 급증



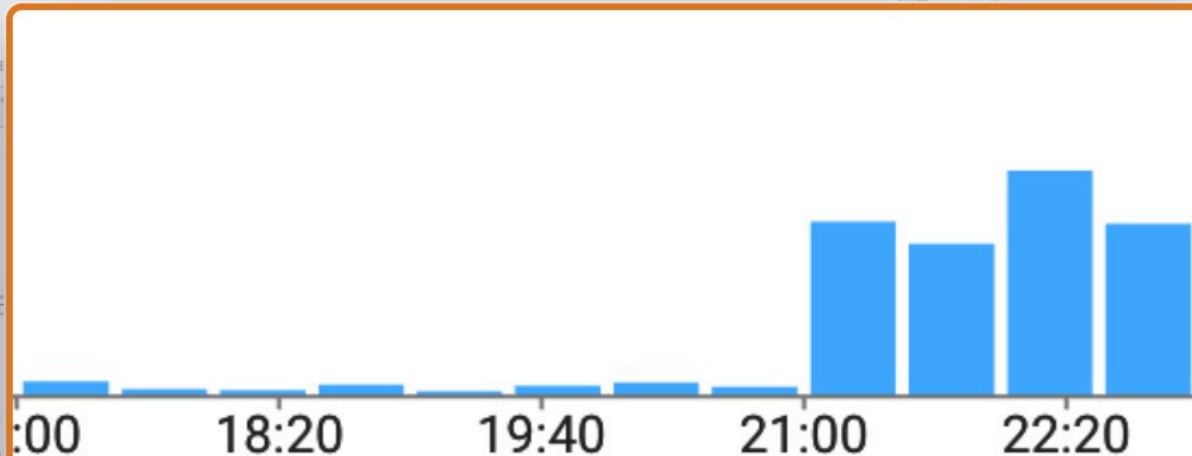
배포 한시간 - 로그 패턴을 확인

평시 수십 건 → 짧은 시간에 1,094건. ERROR 로그가 배포 직후 급증

1 / 3

Trend 우측 spike — 배포 직후 급증

평시 비어 있던 ERROR Trend가 우측 짧은 구간에 큰 막대로 몰림



```
2026-05-19 23:22:56.122 io.level1.tammy.event.handler.create
2026-05-19 23:22:56.122 org.springframework.web.client.HttpServer
2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
2026-05-19 23:22:56.122 org.springframework.dao.DataIntegrityViolationException: could not execute statement; SQL [INSERT INTO orders ...]
2026-05-19 23:22:56.122 org.springframework.web.client.HttpServerErrorException$InternalServerError: 500 Internal Server Error: "{\"code\":\"PG_TIMEOUT\"}"
2026-05-19 23:22:56.122 java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 30000ms
2026-05-19 23:22:56.122 java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 30000ms
2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
2026-05-19 23:22:56.122 java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 30000ms
2026-05-19 23:22:56.122 java.util.concurrent.TimeoutException: Did not observe any item or terminal signal within 5000ms in 'flatMap' (and no fallback)
2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
```


배포 한시간 - 로그 패턴을 확인

평시 수십 건 → 짧은 시간에 1,094건. ERROR 로그가 배포 직후 급증

3 / 3

에러 유형 — 외부·연결 문제 위주

Timeout / HikariPool / PG_TIMEOUT /
Connection refused — 외부 의존 패턴이
같이 튀어



```
InternalServerError: 500 Internal Server Error: "{\"code\":\"PG_TIMEOUT\"}"
expired (couponId=CPN-99213, userId=U-2018)
could not execute statement; SQL [INSERT INTO orders ...]
InternalServerError: 500 Internal Server Error: "{\"code\":\"PG_TIMEOUT\"}"
- Connection is not available, request timed out after 30000ms
```

```
order-api-2 2026-05-19 23:22:56.122 java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 30000ms
order-api-2 2026-05-19 23:22:56.122 java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 30000ms
order-api-2 2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
order-api-2 2026-05-19 23:22:56.122 java.util.concurrent.TimeoutException: Did not observe any item or terminal signal within 5000ms in 'flatMap' (and no fallback)
order-api-2 2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
order-api-2 2026-05-19 23:22:56.122 io.whatap.demo.exception.CouponInvalidException: coupon expired (couponId=CPN-99213, userId=U-2018)
```

“어제 배포 때문이야” — 아닌데?

발견된 날이 발생한 날이 아닐 수 있다

14일 전

어제

오늘

← 14일 동안 묻혀 있음 →

← 1일 →

최초 발생

낮은 빈도
묻혀 있던 에러

배포

의심받음

발견

임계치 도달

발견 ≠ 발생

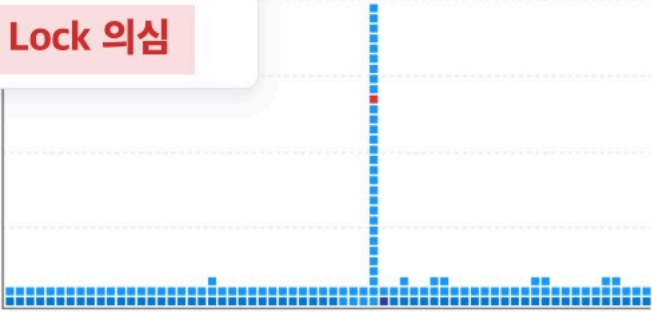
발견 시점과 발생 시점은 다르다 — 최근 14일 에러 발생 타임라인으로 먼저 확인.

그래프 모양 자체가 진단입니다

원인 후보를 좁히는 신호 — 진단이 아니라 시작점

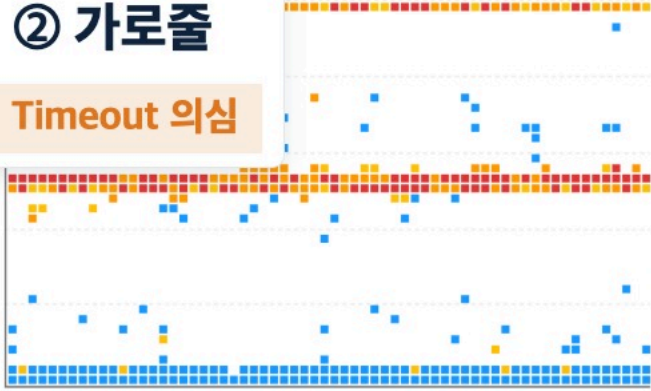
① 세로줄

Lock 의심



② 가로줄

Timeout 의심



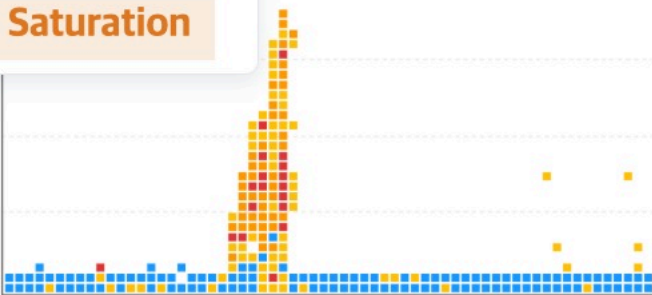
③ 플라잉

주기 자원



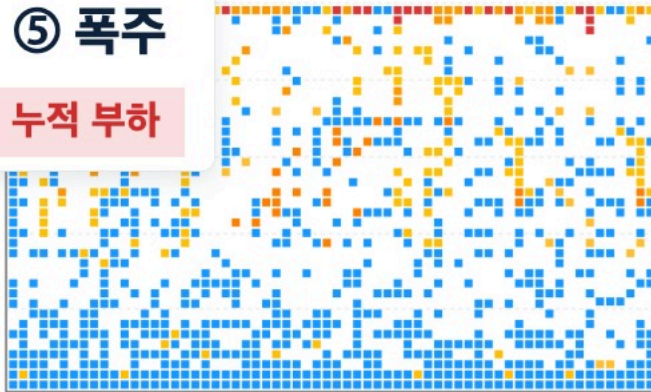
④ 과부하

Saturation



⑤ 폭주

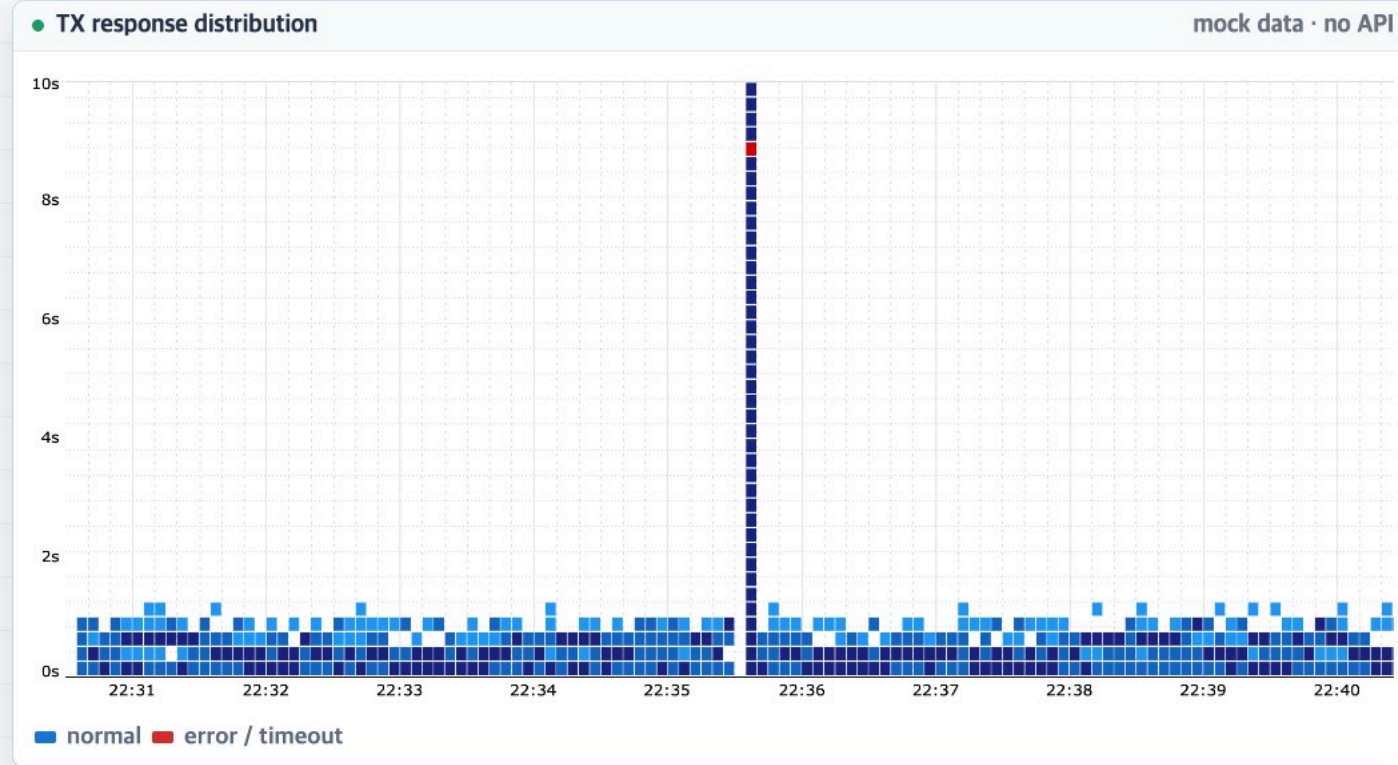
누적 부하



세로줄 — Lock 의심

의심: 공통 대기 해소로 동시 종료 / 확인:
active stack에서 공통 method·SQL 비교

Live heatmap mock · lock



다음 액션

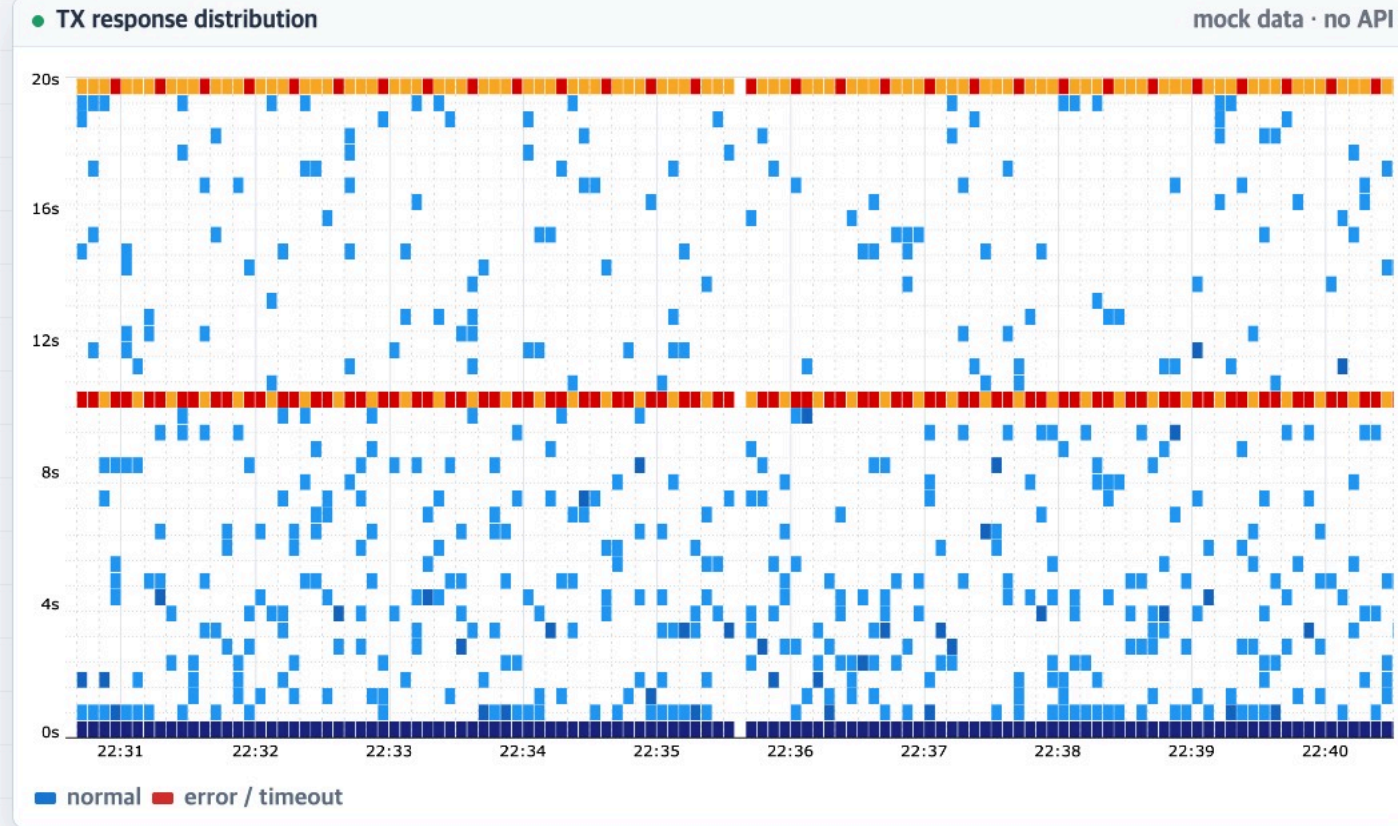
가설: lock release 또는 공통 대기 해소
로 한꺼번에 종료

확인: 최장·최단 트랜잭션의 active
stack에서 공통 method·SQL 비교

가로줄 — Timeout 반복

응답시간이 특정값(예: 10초)에 묶임

Live heatmap mock · timeout



다음 액션

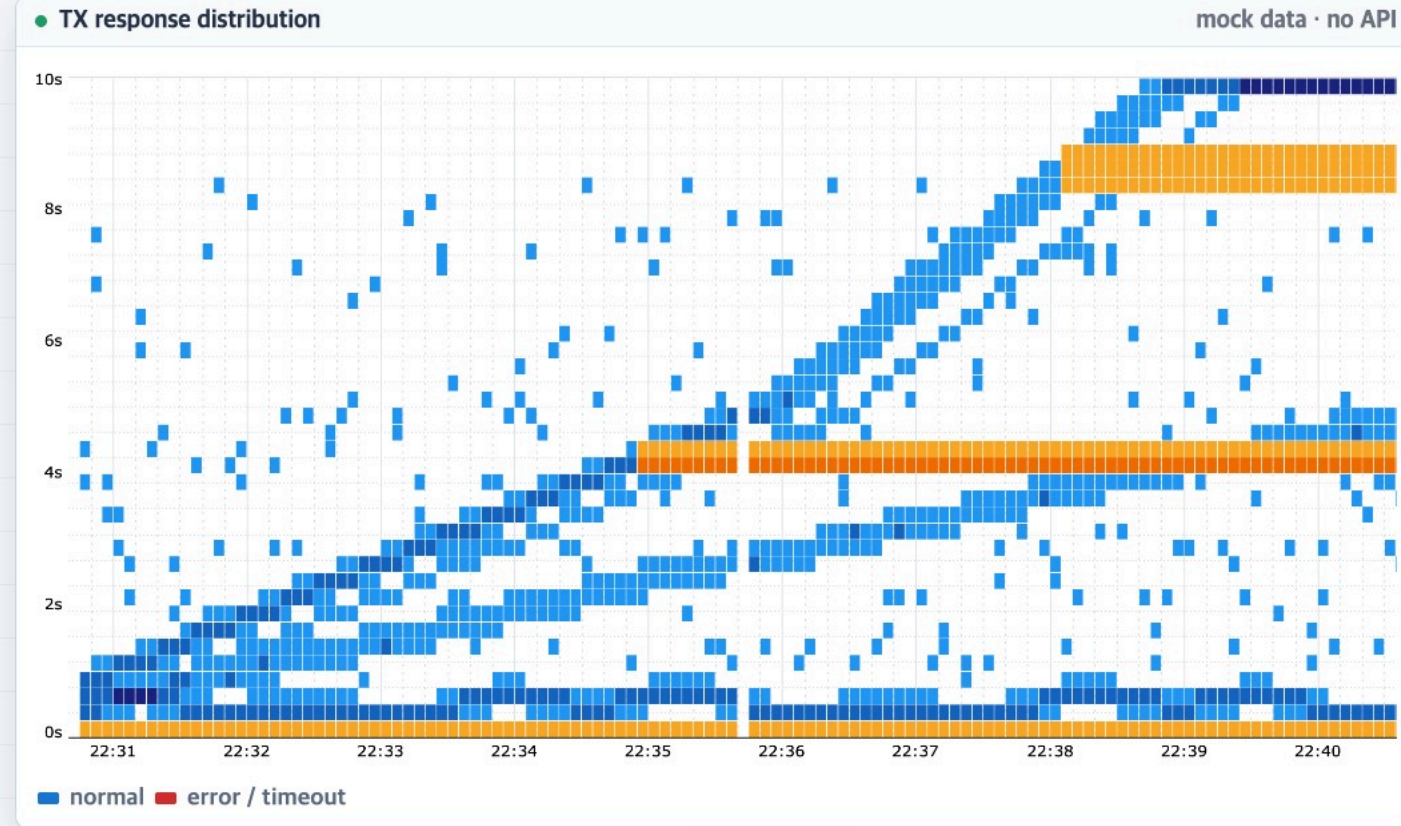
재시도 로직과 외부 의존 서비스 점검

Timeout 값 자체가 의심: 적정한가

플라잉 — 간헐적 자원 부족

원인 확정 아님 — 같은 주기의 배치·GC·스케줄러와 시간축 대조

Live heatmap mock · flying



다음 액션

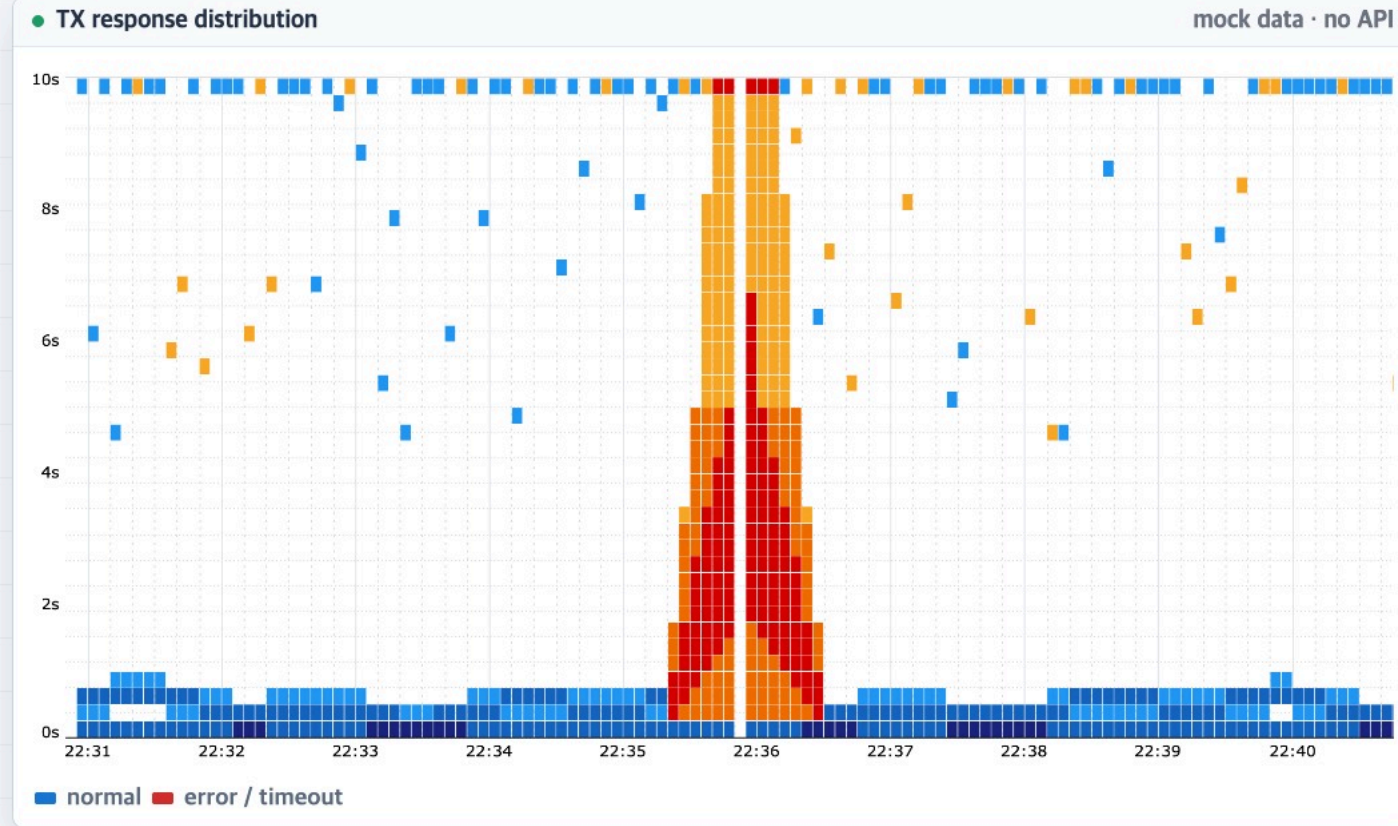
주기적 응답시간 상승 구간을 먼저 표시

같은 주기 배치 · 스케줄러 · GC 로그와
시간축 정렬

과부하 — 동시 폭증 + 일부 실패

응답시간 분포 전체가 위로 퍼지고, 우측에 error/timeout 샘플이 붙음

Live heatmap mock · overload



다음 액션

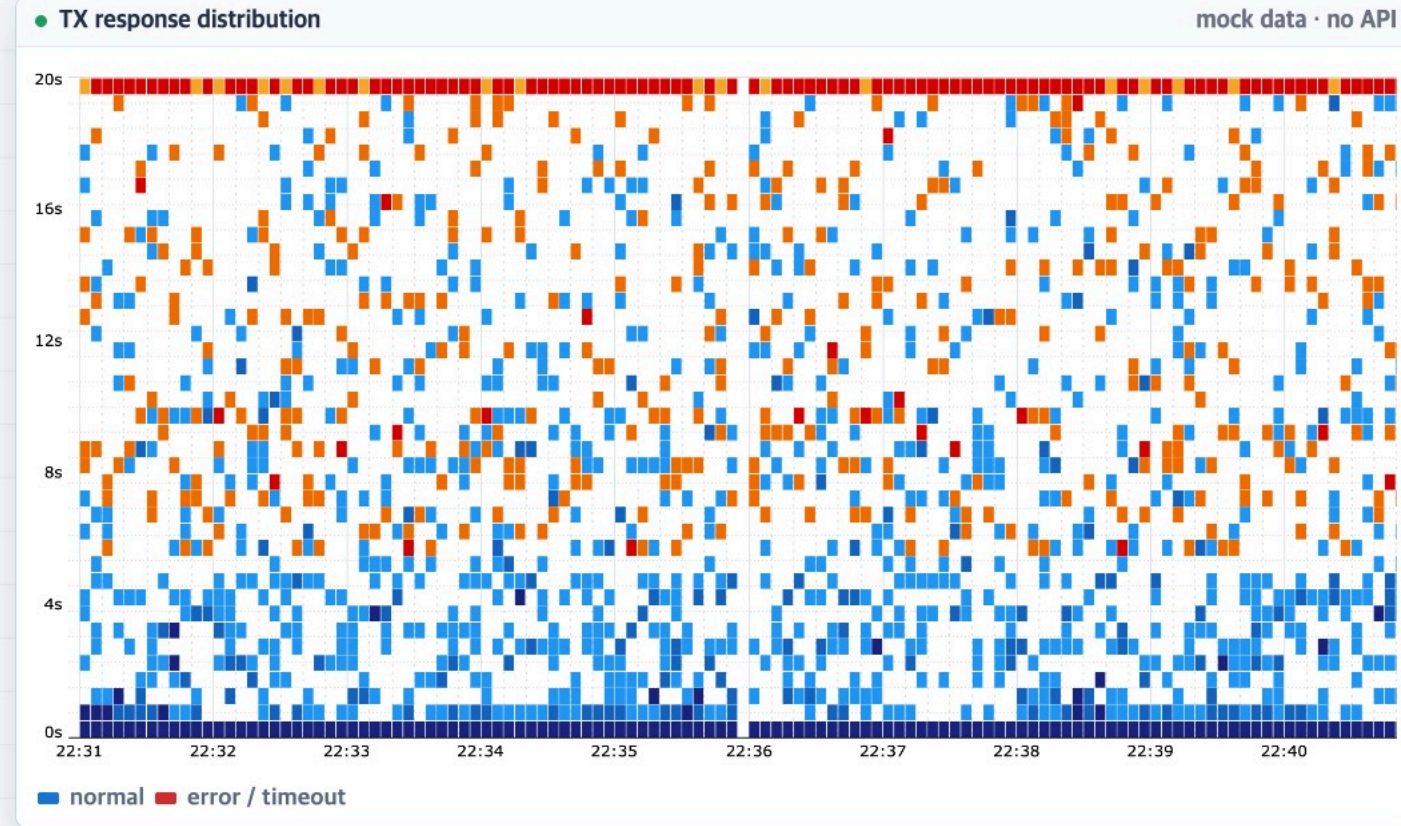
폭증 구간 / 일부 실패 라벨 확인

TPS · P95 · 오류율 · 인스턴스별 편중
같이 보기

폭주 — 누적된 로드

timeout에 붙은 요청이 지속 = 처리 지연 누적

Live heatmap mock · runaway



다음 액션

상단 20s 라인에 요청이 계속 붙는지 확인

SQL / Methods 반복 호출 점검 · 필요시 트래픽 차단/격리

서비스가 커지면 드러나는 것들

코드는 안 바꿨는데 — 하나씩 문제가 발생합니다. 서비스가 어느 단계에 있느냐에 따라 다른 문제가 보입니다.

사용자



latency
error

데이터



slow
SQL
N+1

동시성



pool
lock

메모리



GC
heap

처음부터 느린 쿼리는 별로 없다

회원이 늘고, 데이터가 쌓이면

초기
응답

20_{ms}

아이템 10개 · 회원 100명 · 데이터 적음



코드는 그대로
데이터·회원 100x 증가

성장 후
응답

3,200_{ms}

아이템 1,000개 · 회원 100,000명

SQL 1,001회 · DB time 3,000ms (Trace에서 확인)

처음부터 느린 쿼리는 별로 없다

회원이 늘고, 데이터가 쌓이면

APM

초기
응답
20ms

아이템 10개 · 회원 100명 · 데이터 적음

데이터·회원 100x 증가

성장 후
2,200ms

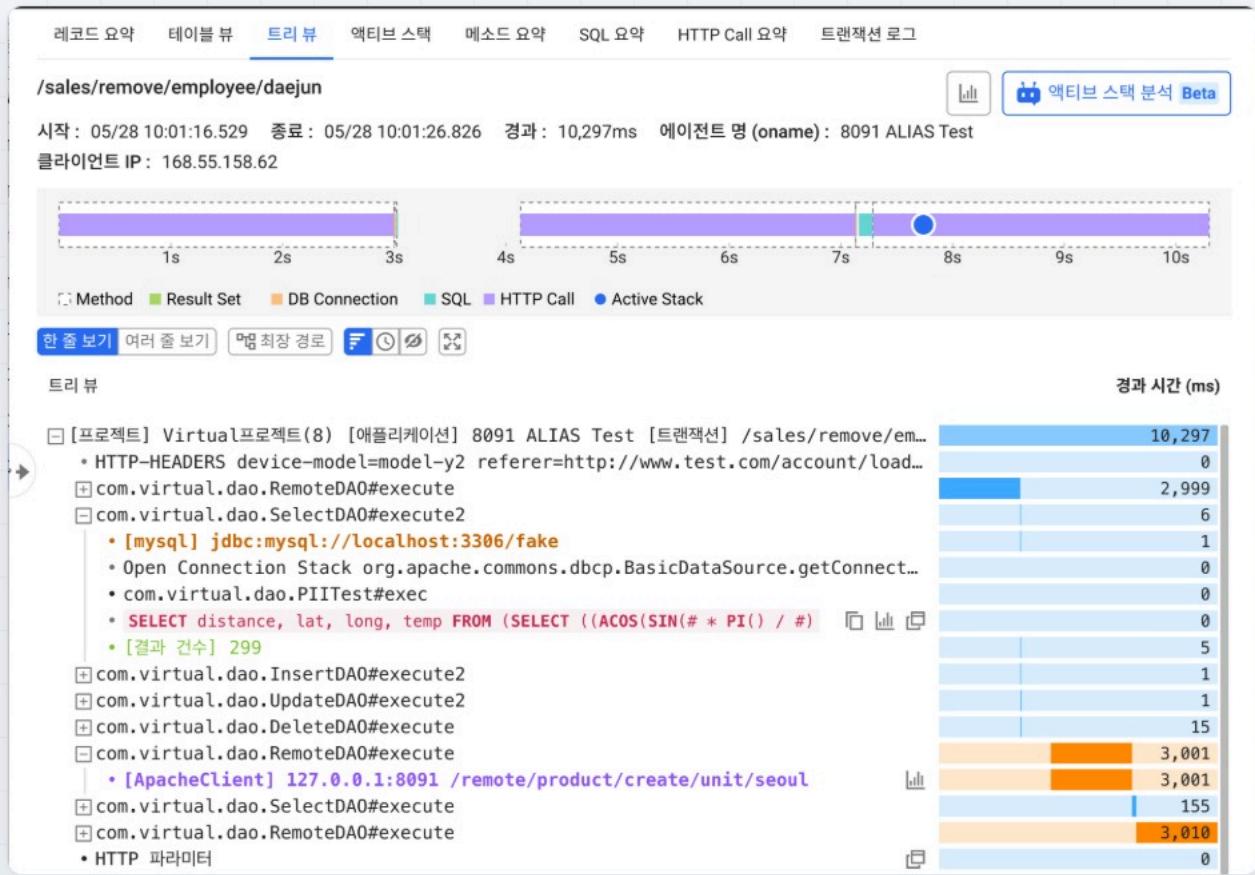
아이템 1,000개 · 회원 100,000명

SQL 1,001회 · DB time 3,000ms (Trace에
기록됨)

Application Performance Monitoring · 트레이스 분석으로 잡는다

트레이스는 요청이 지나간 길입니다

한 줄 = 한 구간 · 들여쓰기 = 계층 · 막대 = 시간

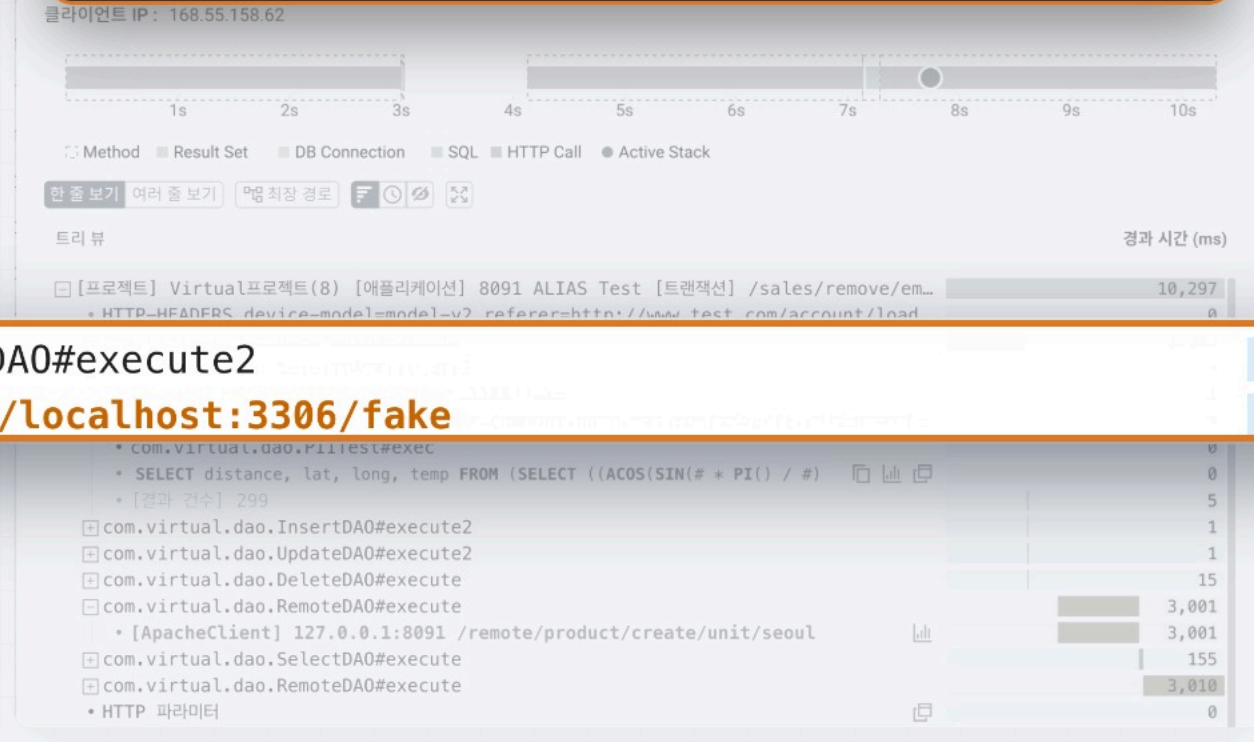


트레이스는 요청이 지나간 길입니다 1 / 3

한 줄 = 한 구간 · 들여쓰기 = 계층 · 막대 = 시간

요청이 거쳐간 한 구간

한 줄 = 컨트롤러·서비스·쿼리·외부 호출이 차례로



com.virtual.dao.SelectDAO#execute2

- [mysql] jdbc:mysql://localhost:3306/fake

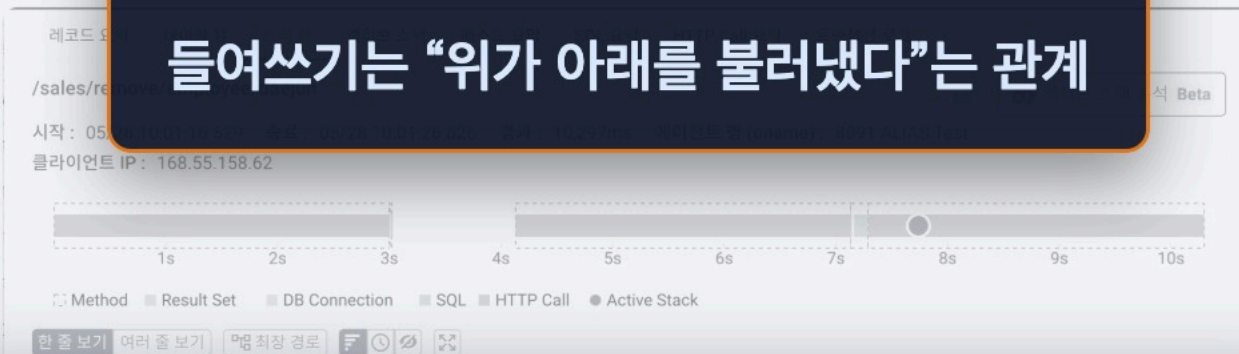
트레이스는 요청이 지나간 길입니다

한 줄 = 한 구간 · 들여쓰기 = 계층 · 도

2 / 3

호출이 서로를 감싸는 계층

들여쓰기는 “위가 아래를 불러냈다”는 관계



[-] com.virtual.dao.SelectDAO#execute2

- [mysql] jdbc:mysql://localhost:3306/fake
- Open Connection Stack org.apache.commons.dbcp.BasicDataSource.getConnection...
- com.virtual.dao.PIIITest#exec
- **SELECT distance, lat, long, temp FROM (SELECT ((ACOS(SIN(# * PI()) / #))**
- [결과 건수] 299

[+] com.virtual.dao.InsertDAO#execute2

[+] com.virtual.dao.UpdateDAO#execute2

[+] com.virtual.dao.DeleteDAO#execute

[-] com.virtual.dao.RemoteDAO#execute

• [ApacheClient] 127.0.0.1:8081 /remote/product/create/unit/coou

[-] com.virtual.dao.RemoteDAO#execute

• HTTP 파라미터

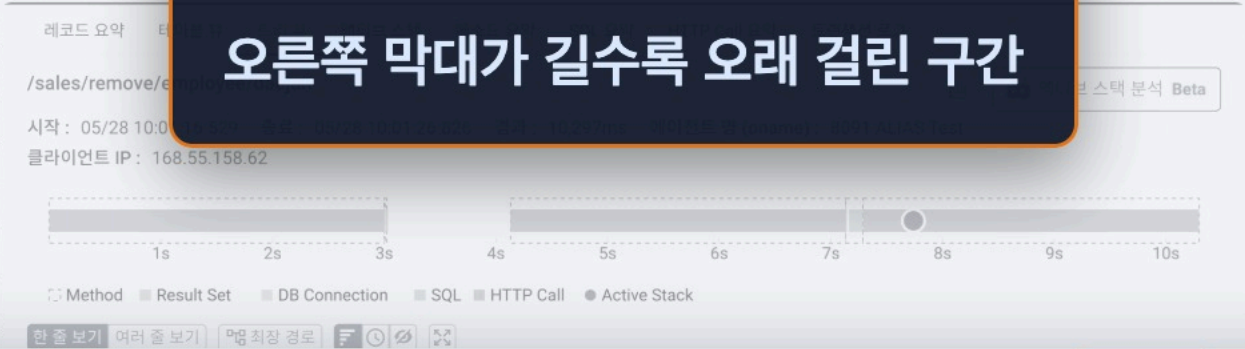
트레이스는 요청이 지나간 길입니다

한 줄 = 한 구간 · 들여쓰기 = 계층 · 막대 = 시간

3 / 3

그 구간에서 걸린 시간

오른쪽 막대가 길수록 오래 걸린 구간



[-] com.virtual.dao.SelectDAO#execute2	6
• [mysql] jdbc:mysql://localhost:3306/fake	1
• Open Connection Stack org.apache.commons.dbcp.BasicDataSource.getConnect...	0
• com.virtual.dao.PIITest#exec	0
• SELECT distance, lat, long, temp FROM (SELECT ((ACOS(SIN(# * PI()) / #)	0
• [결과 건수] 299	5
[+] com.virtual.dao.InsertDAO#execute2	1
[+] com.virtual.dao.UpdateDAO#execute2	1
[+] com.virtual.dao.DeleteDAO#execute	15
[-] com.virtual.dao.RemoteDAO#execute	3,001
• [ApacheClient] 127.0.0.1:8091 /remote/product/create/unit/seoul	3,001

com.virtual.dao.RemoteDAO#execute

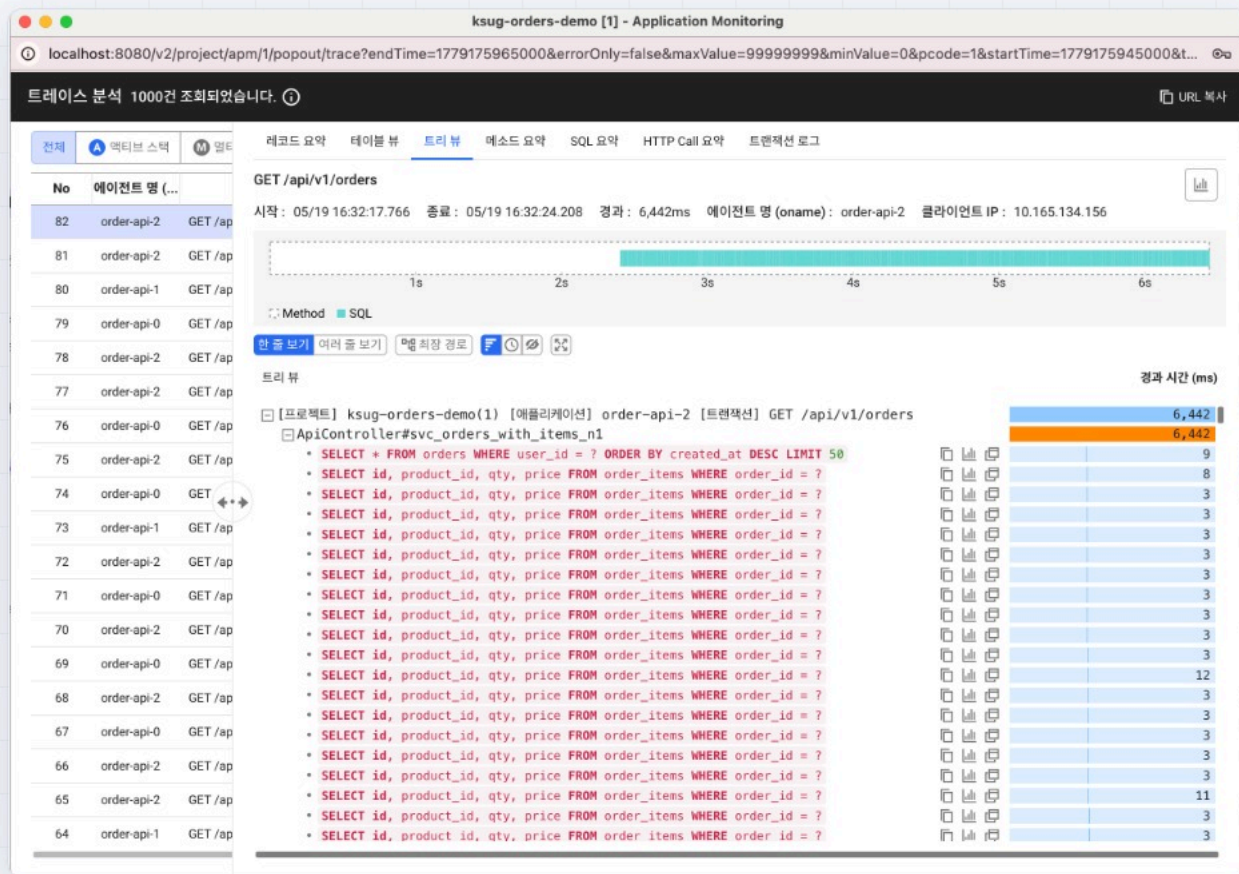
- HTTP 파라미터

3,010

0

코드 한 줄이 트레이스에선 천 줄

ORM 한 줄 = SELECT 1,000번 — 데이터 100배가 곧 SQL 100배



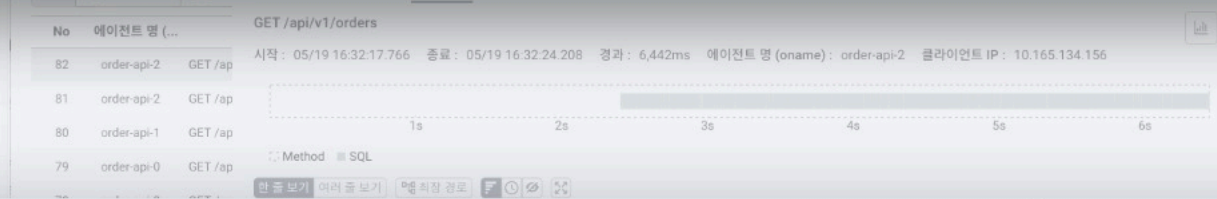
코드 한 줄

ORM 한 줄 = SELECT

트레이스에선 천 줄 1 / 3

ORM 한 줄

ApiController#svc_orders_with_items_n1 — lazy 접근이 트리거



ApiController#svc_orders_with_items_n1

- **SELECT * FROM orders WHERE user_id = ? ORDER BY created_at DESC LIMIT 50**

72	order-api-2	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
71	order-api-0	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
70	order-api-2	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
69	order-api-0	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	12
68	order-api-2	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
67	order-api-0	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
66	order-api-2	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
65	order-api-2	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	11
			* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
64	order-api-1	GET /ap	* SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3

코드 한 줄이 트레이스에선 천 줄

```
[-] ApiController#svc_orders_with_items_n1
```

- [illegible]

2 / 3

SELECT 1,000회 반복

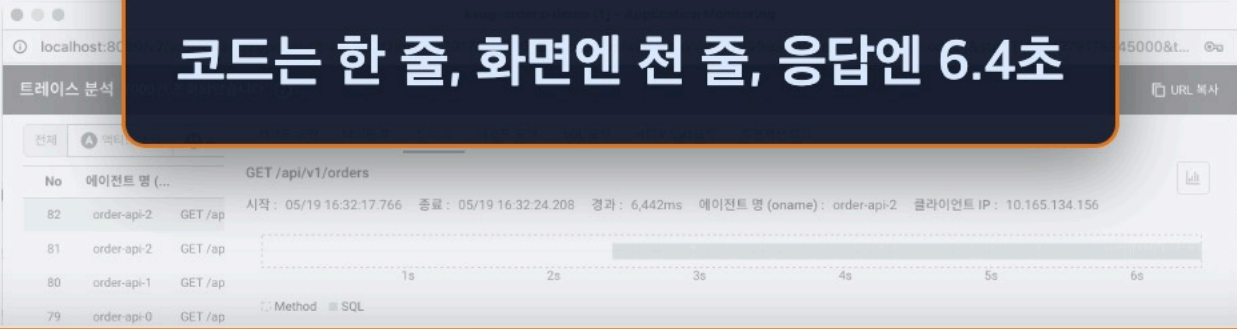
같은 SELECT ... FROM items WHERE id = ?
가 데이터만큼 펼쳐짐

코드 한 줄이 트레이스에선 천 줄 3 / 3

ORM 한 줄 = SELECT 1,000번 — 데이터 1000개

총 응답 6,442ms

코드는 한 줄, 화면엔 천 줄, 응답엔 6.4초



☐ [프로젝트] ksug-orders-demo(1) [애플리케이션] order-api-2 [트랜잭션] GET /api/v1/orders

☐ ApiController#svc_orders_with_items_n1

- **SELECT * FROM orders WHERE user_id = ? ORDER BY created_at DESC LIMIT 50**
- **SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?**
- **SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?**

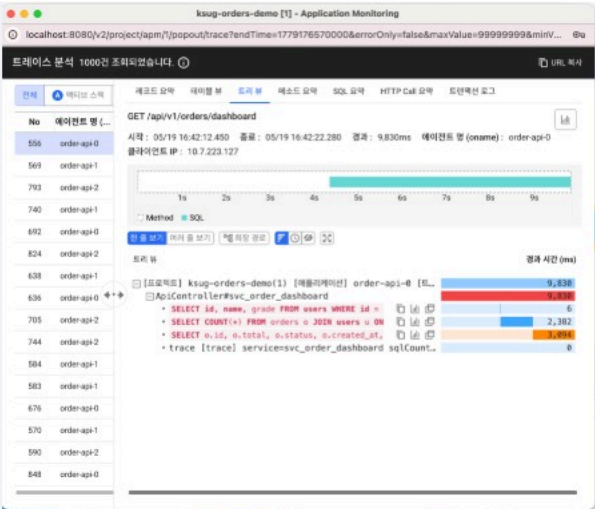
6,4
6,4

70	order-api-2	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
69	order-api-0	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
68	order-api-2	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	12
67	order-api-0	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
66	order-api-2	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
65	order-api-2	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
64	order-api-1	GET /ap	• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	11
			• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3
			• SELECT id, product_id, qty, price FROM order_items WHERE order_id = ?	3

같은 트레이스, 다른 누수

무거운 조인 · 레이어별 중복 호출 — 트레이스 한 화면에 다 보입니다

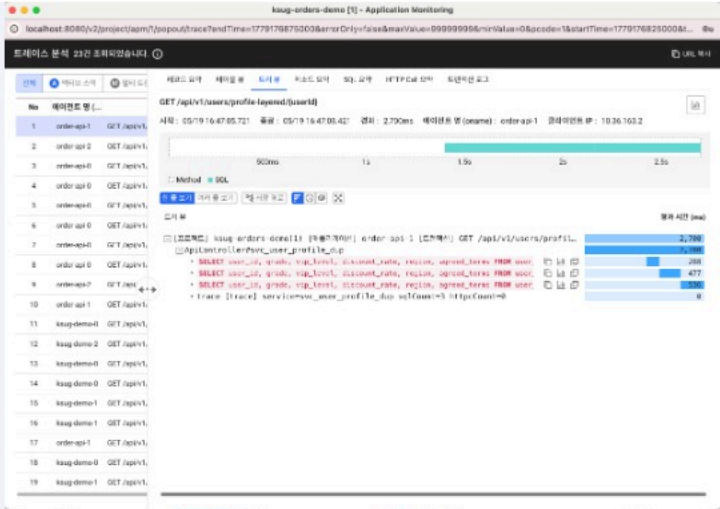
케이스 A 6테이블 조인



2.4s + 3.9s

COUNT + 6테이블 SELECT · 응답 9.8s

케이스 B 같은 SQL 3회



× 3

Controller · Service · Repository 각자 조회 · 응답 2.7s

코드는 깔끔해도, 트레이스에는 누수가 보입니다.

응답시간 3초 → 0.3초

그대로 면접 답이 된다

Before

응답 p95

3.0_s



Trace에서 느린 SQL · N+1 발견
→ 쿼리 개선

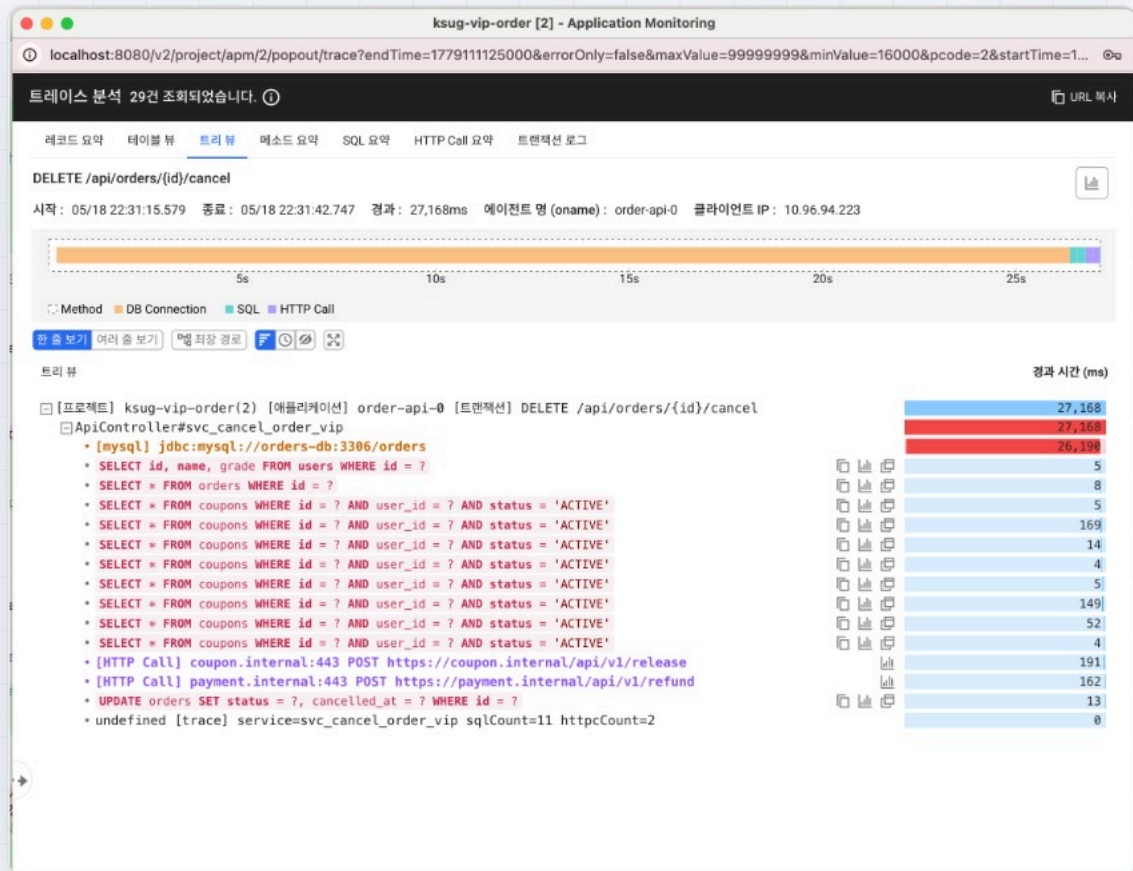
After

응답 p95

0.3_s

평소 0.5초 API가 27초

커넥션 풀 대기 — SQL은 빠르는데, 전체가 27초



평소 0.5초 API가 27초

1 / 3

커넥션 풀 대기 — SQL은 빠르는데, 전체가

DB Connection 26.2초

mysql jdbc 획득 대기 — 거의 전부가 여기에 묶임



ApiController#svc_cancel_order_vip

• [mysql] jdbc:mysql://orders-db:3306/orders

27,168

26,190

```
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
• [HTTP Call] coupon.internal:443 POST https://coupon.internal/api/v1/release
• [HTTP Call] payment.internal:443 POST https://payment.internal/api/v1/refund
• UPDATE orders SET status = ?, cancelled_at = ? WHERE id = ?
• undefined [trace] service=svc_cancel_order_vip sqlCount=11 httpCount=2
```



평소 0.5초 API가 27초

커넥션 풀 대기 — SQL은 빠르는데,

2 / 3

실제 SQL · HTTP는 1초 미만

5~191ms짜리 쿼리·HTTP가 합쳐도 1초가 안 됨

DELETE /api/orders/{id}/cancel

시작: 05/18 22:31:15.579 종료: 05/18 22:31:42.747 경과: 27,168ms 에이전트명 (oname): order-api-0 클라이언트 IP: 10.96.94.223



Method DB Connection SQL HTTP Call

원래 보기 여러 줄 보기 에러 저장 경로

ApiController#svc_cancel_order_vip

- [mysql] jdbc:mysql://orders-db:3306/orders

- SELECT id, name, grade FROM users WHERE id = ?
- SELECT * FROM orders WHERE id = ?
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'
- SELECT * FROM coupons WHERE id = ? AND user_id = ? AND status = 'ACTIVE'

	27,168
	26,190
	5
	8
	5
	169
	14
	4
	5
	149
	52
	4

평소 0.5초 API가 27초

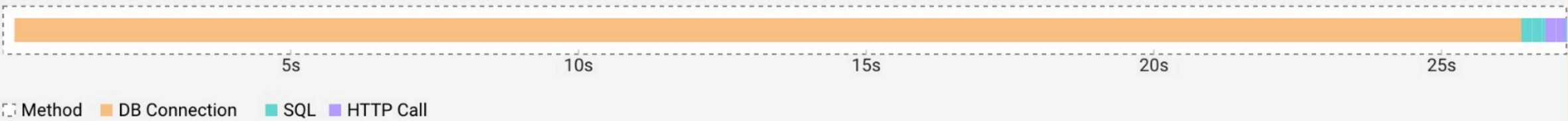
커넥션 풀 대기 — SQL은 빠르는데, 전체가 27초

3 / 3
전체 응답 27.2초
쿼리는 빠르는데, 전체는 27초



DELETE /api/orders/{id}/cancel

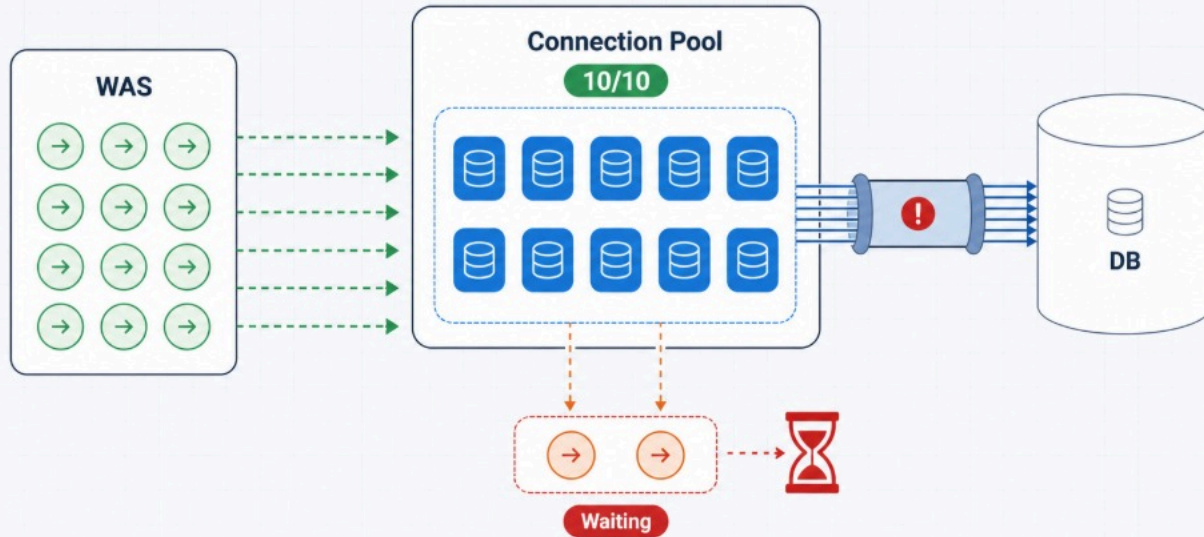
시작: 05/18 22:31:15.579 종료: 05/18 22:31:42.747 경과: 27,168ms 에이전트 명 (oname): order-api-0 클라이언트 IP: 10.96.94.223



커넥션 풀 = DB로 가는 통로 개수

Spring Boot의 HikariCP — 기본

`maximumPoolSize=10`, 11번째 요청부터 줄선다



원인

동시 요청이 풀(기본 10)을 넘으면 줄 선다 —
커넥션 대기 시작

결과

SQL은 1ms, 요청은 수십 초 — 슬로우 쿼리
로그엔 안 잡힘

특정 API가 아니라, 전부 느려졌다

트레이스가 답 못 할 때 — 어디를 봐야 할까

CPU 100%



Error 0건



Trace 원인 불명

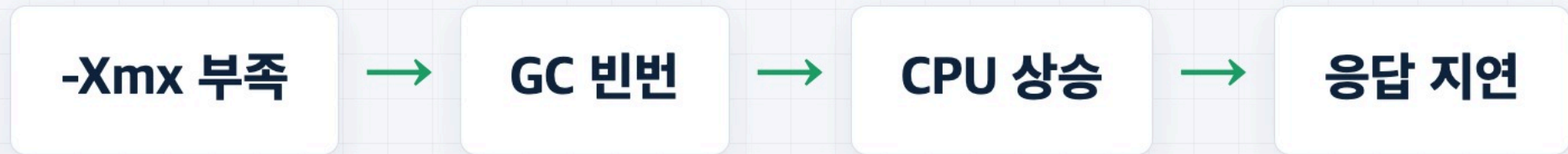


Server Metric 확인

트레이스만 봐선 안 보입니다 — 서버 메트릭으로 가야 합니다.

힙 메모리 = JVM 객체 저장 공간

-Xmx 부족 → GC 빈번 → CPU 상승 → 응답 지연



힙이 차면 GC가 CPU를 먹습니다 — 그리고 그 GC가 다음 장에서 트레이스에 안 잡힙니다.

어떤 메소드도 “내가 느렸다”라고 말하지 않는다

GC는 메소드 사이에 끼어들어 일하니까 — 트레이스 어디에도 안 잡힌다



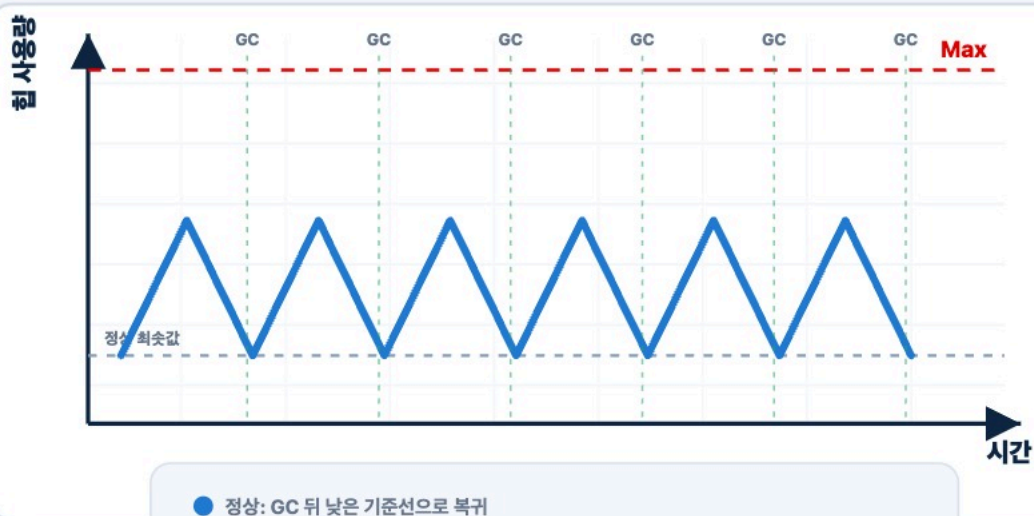
트레이스에 안 잡히는 800ms — “코드는 빠르는데 왜 느리지?”의 정체

최솟값이 계속 올라가면

GC 후 바닥선 상승 → 히스토그램 → 힙 덤프 retained 추적

메모리 누수 패턴

GC 후에도 사용량 최솟값이 원래 수준으로 내려오지 않는다



신호

GC 후에도 **live set 바닥선**이 우상향 = 누수 의심

진단

- ① 클래스 히스토그램에서 비정상 인스턴스 →
- ② 힙 덤프에서 **retained size · GC root · reference path**

코드 변경 없이도 시간이 흐르면 드러나는 누수 — 1시간이 아니라 다음날 · 다음주에 잡힙니다

최솟값이 계속 올라가면

GC 후 바닥선 상승 → 히스토그램 → 힙 덤프 retained 추적



신호

GC 후에도 **live set 바닥선**이 우상향 = 누수 의심

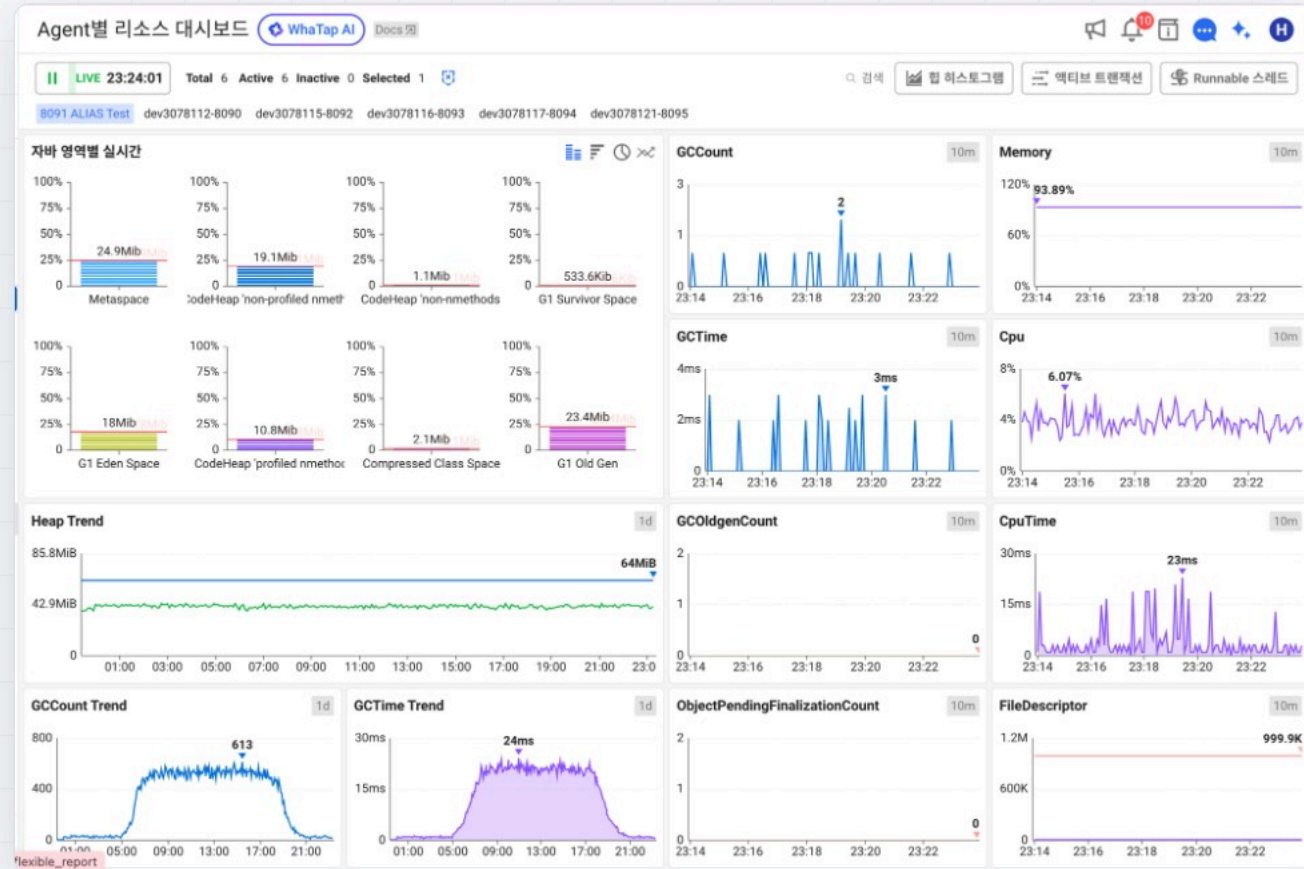
진단

- ① 클래스 히스토그램에서 비정상 인스턴스 →
- ② 힙 덤프에서 **retained size · GC root · reference path**

코드 변경 없이도 시간이 흐르면 드러나는 누수 — 1시간이 아니라 다음날 · 다음주에 잡힙니다

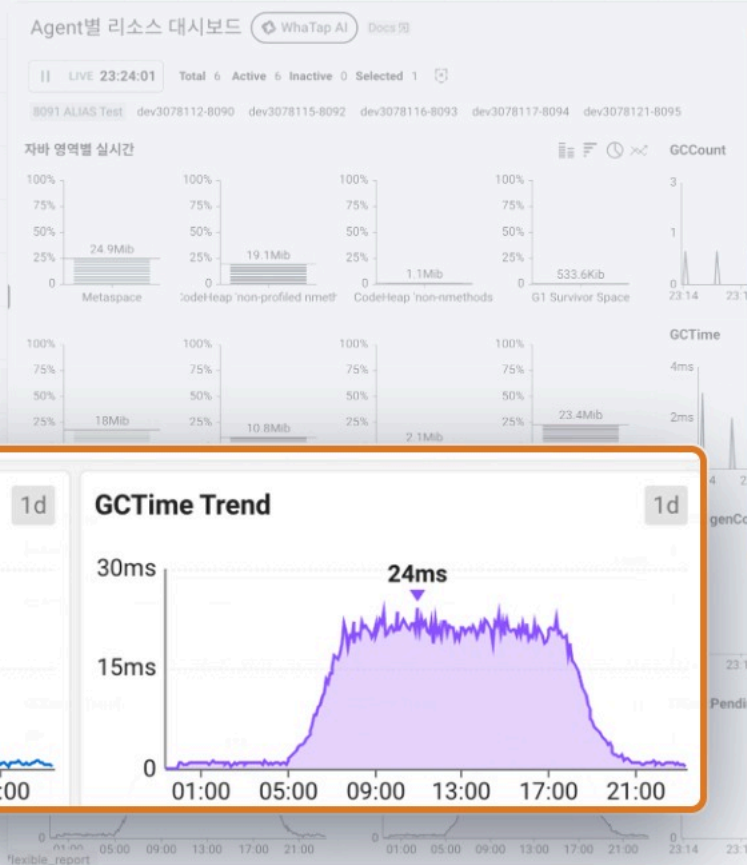
같은 화면에 겹쳐 보면 — 결론이 나온다

GC · CPU · Heap 같은 시간축에 정렬



같은 화면에 겹쳐 보면 — 결론이 나온다

GC · CPU · Heap 같은 시간축에 정렬



1 / 3

GC가 도는 시점 — 업무 시간 집중

GCCount · GCTime 트렌드가 같은 시간대에 동반 상승

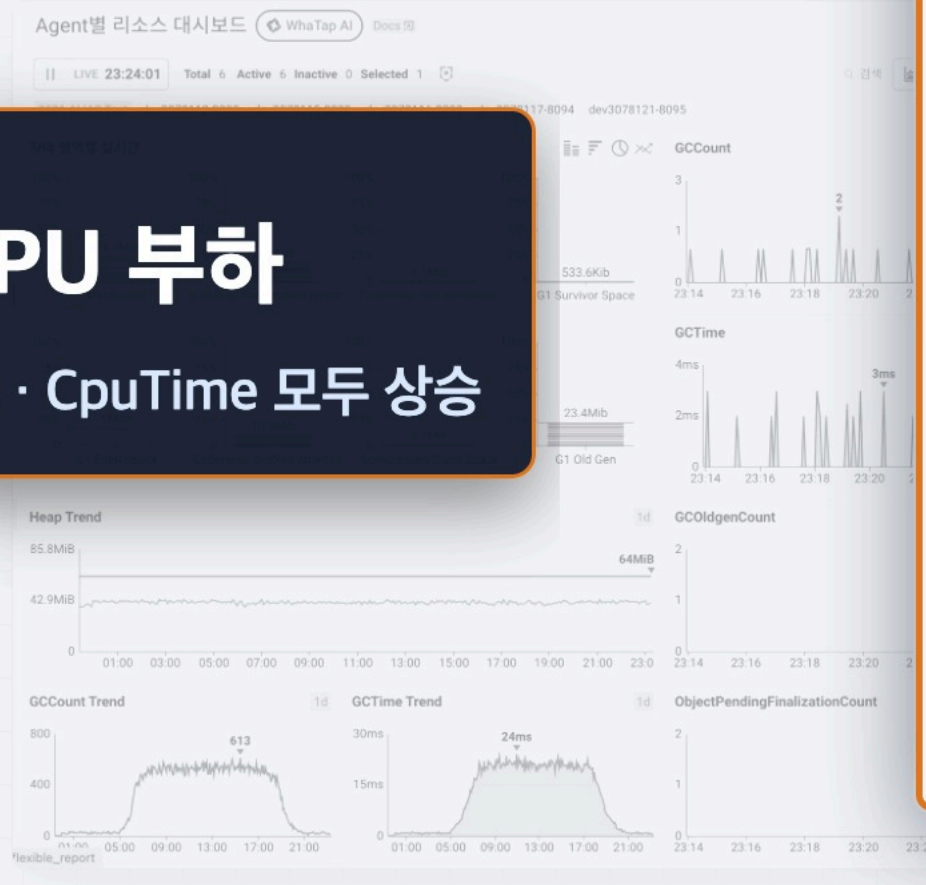
같은 화면에 겹쳐 보면 — 결론이 나온다

GC · CPU · Heap 같은 시간축에 정렬

2 / 3

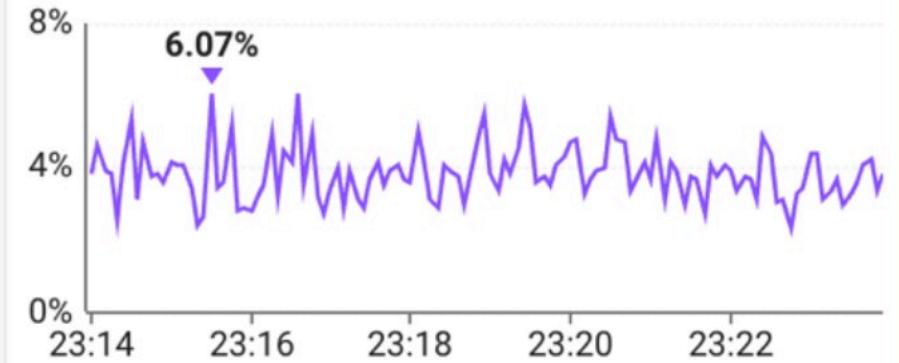
같은 시간대 CPU 부하

GC 활발한 시간에 CPU · CpuTime 모두 상승



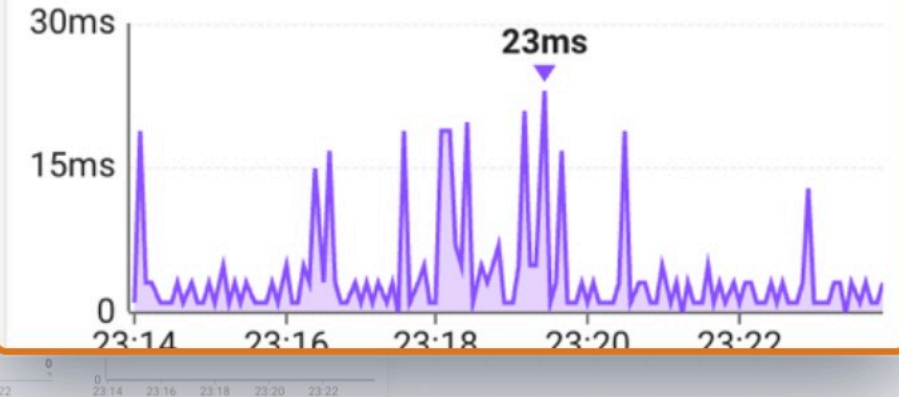
Cpu

10m



CpuTime

10m



같은 화면에 겹쳐 보면 — 결론이 나온다

GC · CPU · Heap 같은 시간축에 정렬

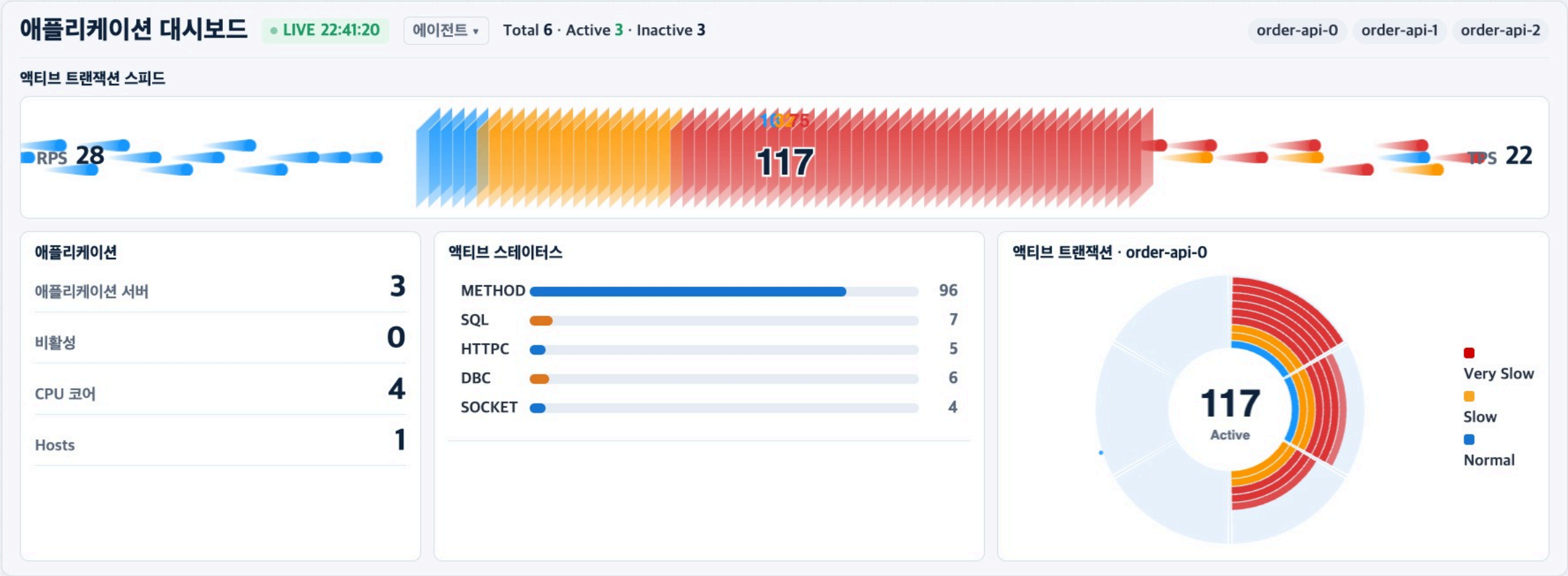


3 / 3

Heap 베이스라인 추세
1일 추세에서 Heap 사용량 패턴 확인

무한 루프 · 무거운 직렬화는?

재현 없이 — 10초마다 자동 스레드 덤프



클라우드 환경일 때

내 코드가 느린 게 아니라, 인프라 때문일 수 있다

신호 CPU Steal ↑

CPU Steal

이웃 VM이 점유 → 내 vCPU 스케줄링 못 받음

VM-A (이웃)

RUN

RUN

RUN

RUN

Me (내 VM)

STEAL

RUN

STEAL

RUN

STEAL

RUN

STEAL

시간

내 코드는 가만히 있는데 — Steal 시간만큼 응답이 밀린다

확인: `vmstat` 의 `st` 컬럼 / `top` · `/proc/stat`

vCPU 대기

하이퍼바이저가 내 vCPU를 바로 스케줄링하지 못한 대기 시간

신호 Disk I/O Throttle ↑

Disk I/O Throttle

IOPS · 대역폭 한도 도달 → 요청 대기 큐 누적

실제 처리량

IOPS 한도

처리량이 한도에 붙어 있음 (포화)

대기 큐

늘어남 ↑

시간

wait time 누적 → 같은 코드도 응답이 느려진다

확인: `CloudWatch BurstBalance` · `VolumeQueueLength` / `iostat` 의 `await`

EBS / 디스크 한도

IOPS · 대역폭 한도에 닿으면 wait time 누적

함께 확인: Swap · Load Average — 인프라 메트릭도 합이 프레임의 일부입니다.

DB 진단

APM = “어디가 느린지” · DB = “왜 느린지”

APM

트레이스

SQL 시간

호출 수

DB

EXPLAIN

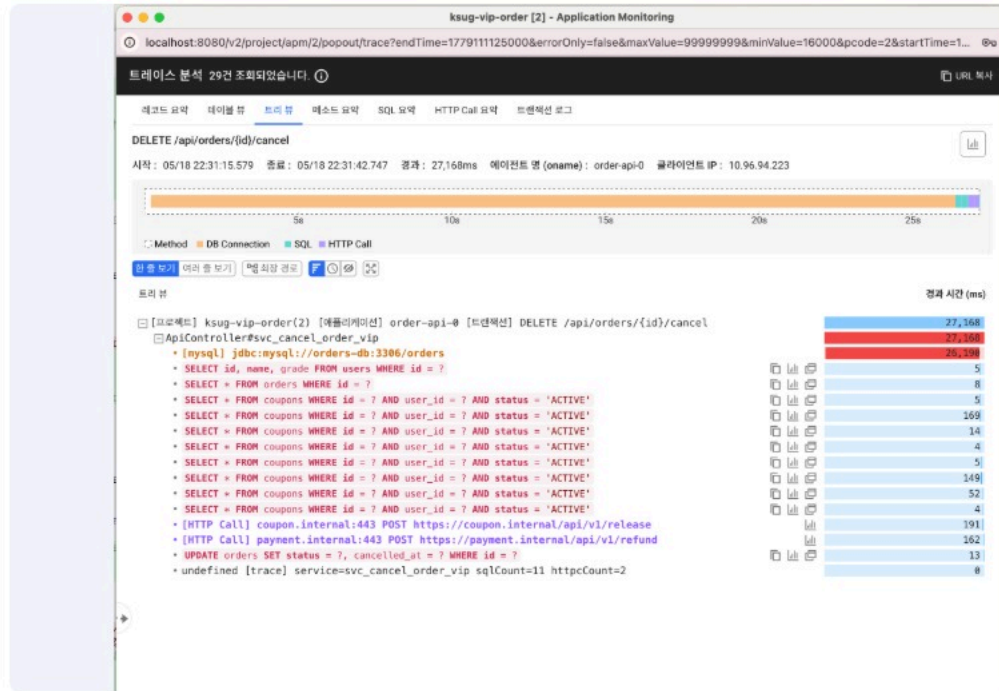
인덱스

락 대기

데이터베이스 모니터링

DB 안에서 무슨 일이 벌어지는지 — Lock · Wait · I/O · Plan

APM 트레이스 어디가 느리지



트레이스 · SQL 시간 · 호출 수

DB 진단 왜 느리지



Active Session · Lock · Wait · I/O · Plan

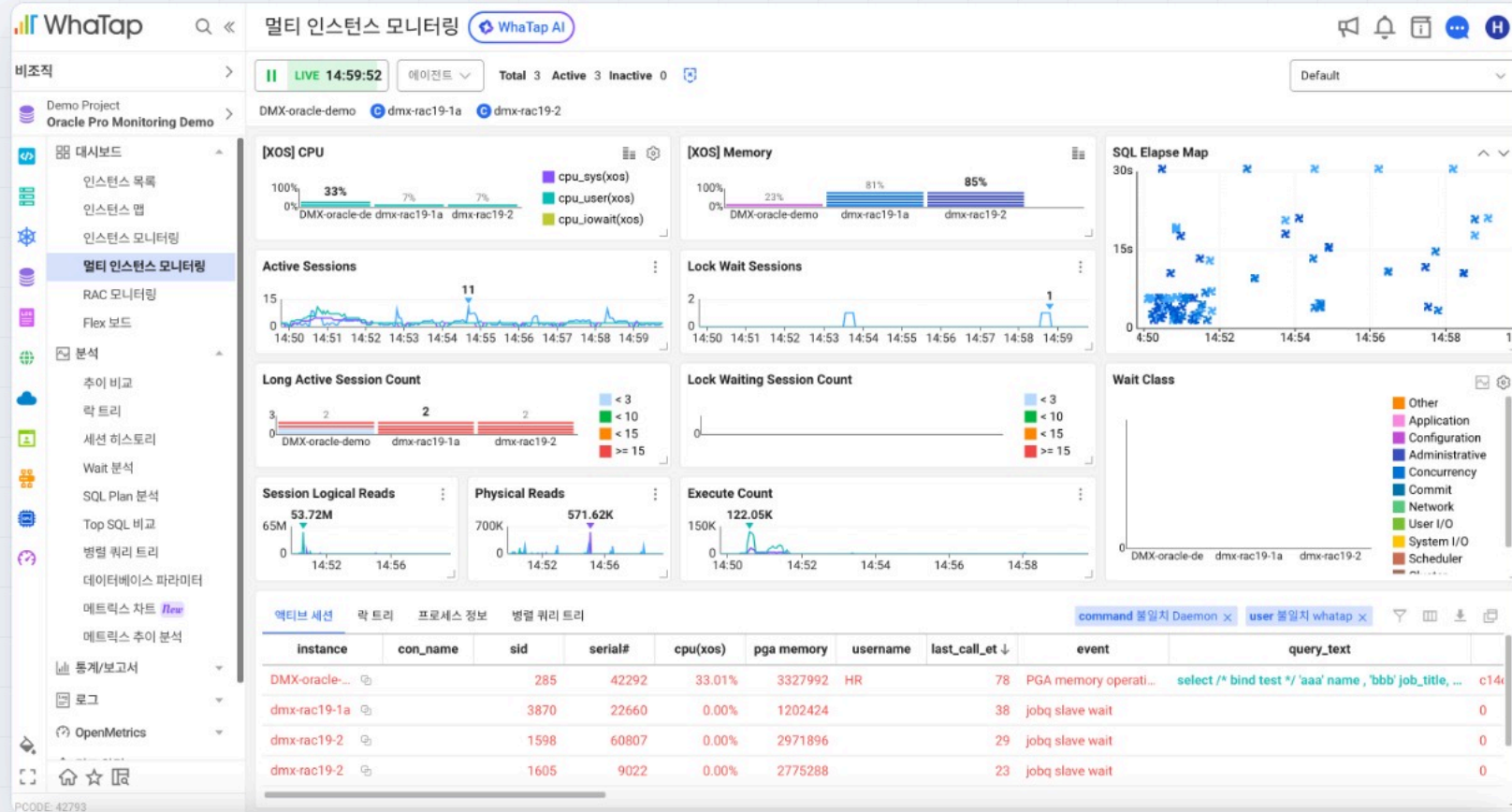
DB 진단 — 일반적인 흐름

한 사건을 끝까지 따라가는 5단계 — 이상 감지부터 원인까지



인스턴스 모니터링 — 전반 1차 스캔

멀티 인스턴스 한 화면 — 튼 시점·지표·드릴다운 후보를 차례로



인스턴스 모니터링 — 전반 1차 스캔

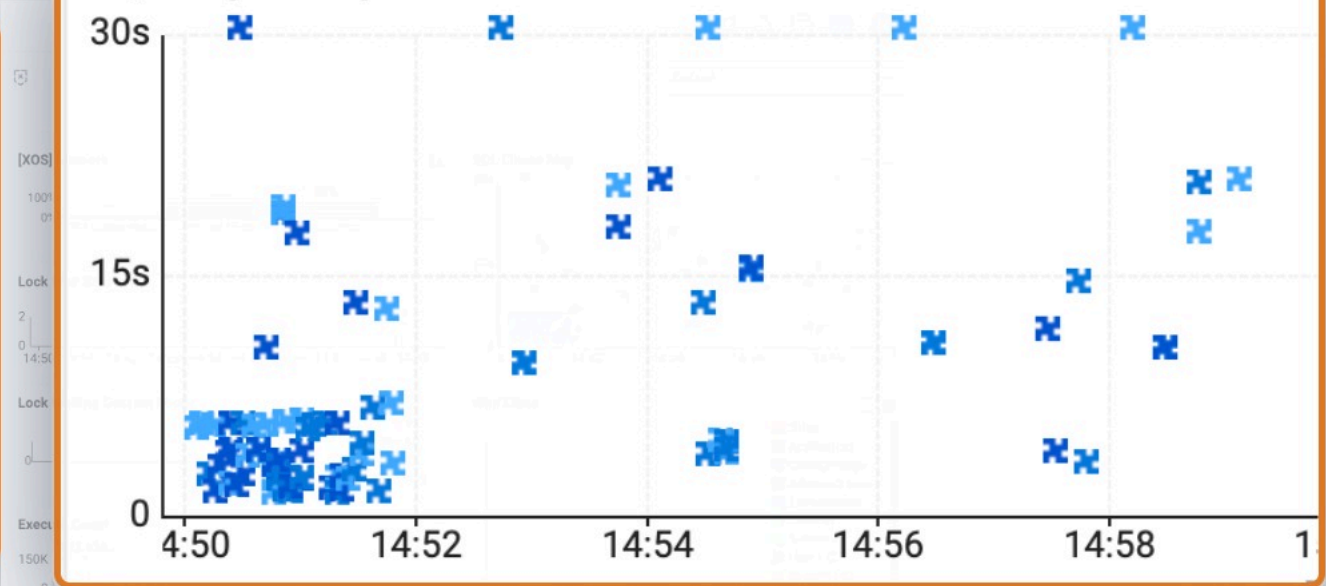
멀티 인스턴스 한 화면 — 튜 시점·지표·드릴다운 후보를 차례로

1 / 3

언제 튀었나 — SQL Elapse Map

튜 시각의 느린 SQL 분포를 잡고, 그 시점을 기준으로 다른 지표와 대조합니다

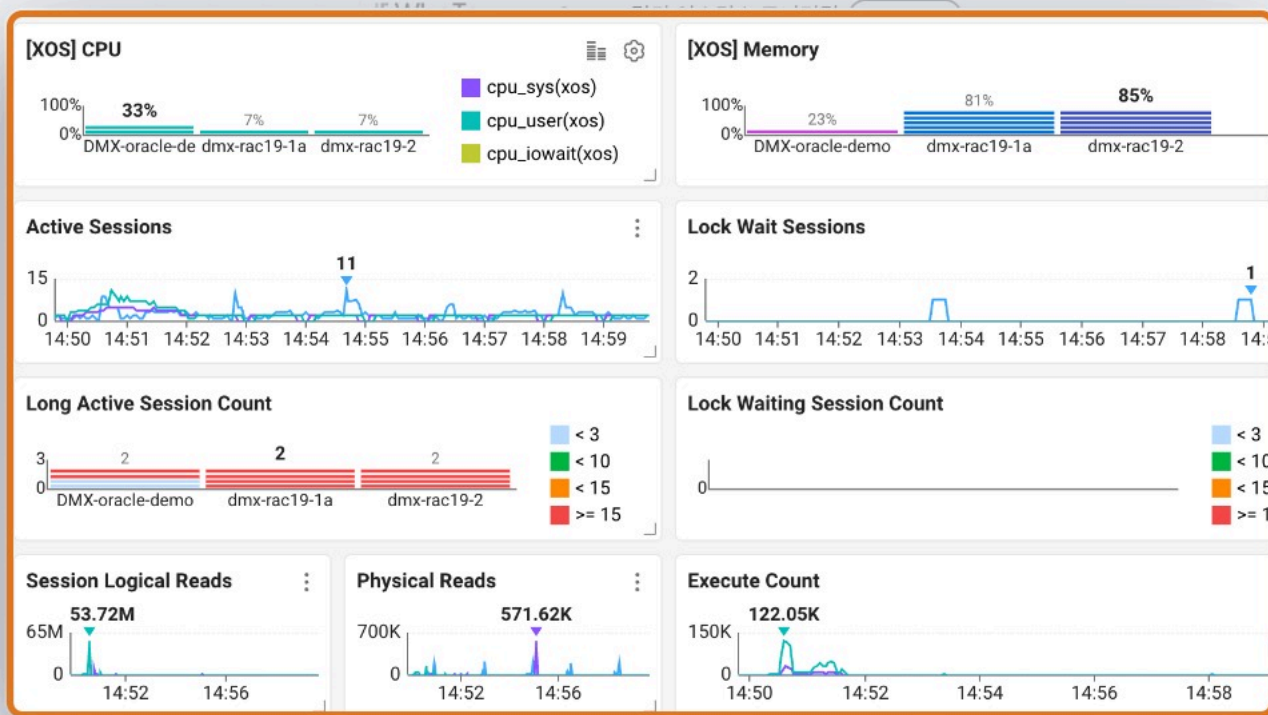
SQL Elapse Map



instance	con_name	sid	serial#	cpu(xos)	pga memory	username	last_call_et ↓	event	query_text
DMX-oracle...		285	42292	33.01%	3327992	HR	78	PGA memory operati...	select /* bind test */ 'aaa' name, 'bbb' job_title, ... c14
dmx-rac19-1a		3870	22660	0.00%	1202424		38	jobq slave wait	
dmx-rac19-2		1598	60807	0.00%	2971896		29	jobq slave wait	
dmx-rac19-2		1605	9022	0.00%	2775288		23	jobq slave wait	

인스턴스 모니터링 — 전반 1차 스캔

멀티 인스턴스 한 화면 — 튼 시점·지표·드릴다운 후보를 차례로



2 / 3

같이 움직인 지표 — CPU·Active·Long Active·Lock

그 시각 CPU·Active Session·Long Active Session·Lock 대기가 함께 튀는지 한 화면에서 대조

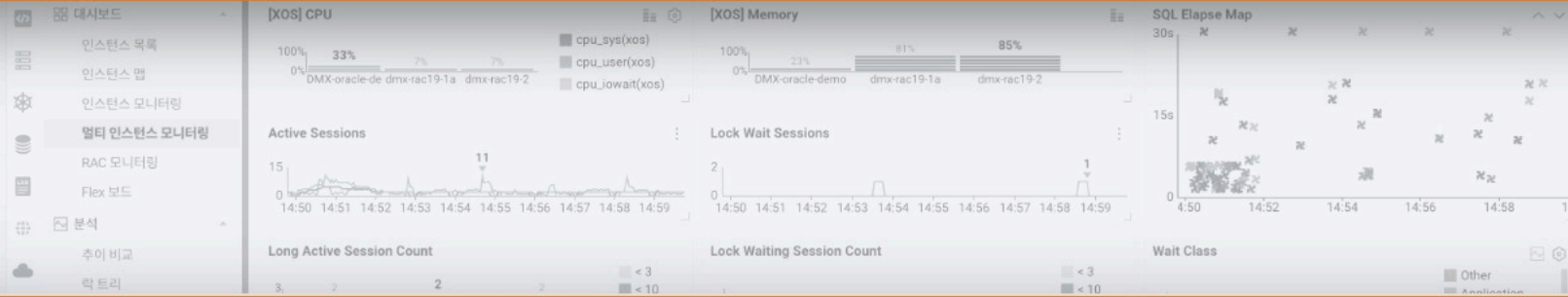
name	last_call_et ↓	event	query_text
78	PGA memory operati...	select /* bind test */ 'aaa' name, 'bbb' job_title, ...	c14
38	jobq slave wait		0
29	jobq slave wait		0
23	jobq slave wait		0

인스턴스 모니터링 — 전반 1차 스텝 3 / 3

멀티 인스턴스 화면 — 원 시점

다음 드릴다운 후보 — 세션·SQL

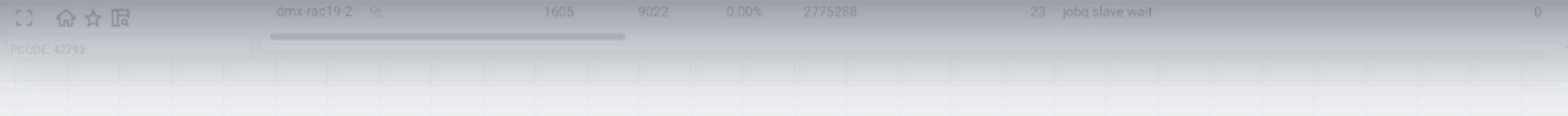
의심 세션/쿼리를 골라 ② 액티브 세션 · ③ 락트리 · ④ SQL Text로 이어 봅니다



액티브 세션 락 트리 프로세스 정보 병렬 쿼리 트리

command 불일치 Daemon × user 불일치 whatap ×

instance	con_name	sid	serial#	cpu(xos)	pga memory	username	last_call_et ↓	event	query_text
DMX-oracle-...		285	42292	33.01%	3327992	HR	78	PGA memory operati...	select /* bind test */ 'aaa' name , 'bbb' job_title, ...
dmx-rac19-1a		3870	22660	0.00%	1202424		38	jobq slave wait	
dmx-rac19-2		1598	60807	0.00%	2971896		29	jobq slave wait	
dmx-rac19-2		1605	9022	0.00%	2775288		23	jobq slave wait	



액티브 세션 — 지금 누가 무엇을 돌리는지

패턴 3개로 의심 방향이 갈린다

① 모니터링 지표

② 액티브 세션

③ 락트리

④ SQL Text

⑤ 플랜

instance	con_name	sid	serial#	cpu(xos)	pga memory	username	last_call_et ↓	event	query_text	sql_id	session logical...	physical reads	exec...
DMX-oracle-demo		52	9934	0.00%	1296376	HR		PL/SQL lock timer	declare aa varchar2(10); bb number; cc numbe...	7h2bvcwptww	0	0	
DMX-oracle-demo		284	27912	15.33%	6194776	SYS		PGA memory operati...	SELECT /*+ procedure_test9 NO_INDEX(e) */ C...	3ws45p33nytd	0	0	
DMX-oracle-demo		267	64882	0.00%	820824			jobq slave wait		0	0	0	
DMX-oracle-demo		61	4087	0.00%	1361912	HR		enq: TX - row lock co...	select /*bind_test*/ from bind_test where regl...	6@vqjma8ubnqz	0	0	
DMX-oracle-demo		285	47768	4.47%	6653528	HR		PX Deq: Execution Msg	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		281	49295	4.47%	6653528	HR		PX Deq: Execution Msg	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		270	53576	5.11%	6653528	HR		PX Deq: Execution Msg	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		64	40643	0.00%	5015128	HR		PX Deq: Table Q Nor...	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		59	51812	0.00%	5015128	HR		PX Deq: Table Q Nor...	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		58	57142	0.31%	5015128	HR		PX Deq: Table Q Nor...	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		49	54917	0.71%	2082808	HR		PX Deq: Execute Reply	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		39	39559	0.00%	5015128	HR		PX Deq: Table Q Nor...	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		29	15532	5.43%	6653528	HR		PX Deq: Execution Msg	select /* parallel test */ count(*) from (select /...	7w42v7np9b4c7	0	0	
DMX-oracle-demo		282	44419	0.31%	10782296	WHATAP		PGA memory operati...	select /* Whatap3D#3 */ l1.type,l1.sid waiter...	3sz91mxg45ntg	0	0	
DMX-oracle-demo		261	13102	0.63%	9602648	WHATAP		PGA memory operati...	select /* Whatap3D#3 */ l1.type,l1.sid waiter...	3sz91mxg45ntg	0	0	
DMX-oracle-demo		54	17713	0.00%	5801560	SHOP		log file sync		0	0	0	

패턴 3개로 다음 단계(③ 락트리 · ④ SQL Text · WAS 조절)가 갑니다.

같은 SQL text 증가

배포 새 쿼리 의심 → ④ 플랜 확인

Lock Waiting + SQL 증가

락 발생 → ③ 락트리 / 원인 세션 확인 후 필요 시 종료

전체 Active Session 급증·지속

DB 리소스 / 이벤트 유입 → WAS 인스턴스 조절

락트리 — holder ↔ waiter

최상위 holder ↔ 영향받는 waiter chain

- ① 모니터링 지표
- ② 액티브 세션
- ③ 락트리
- ④ SQL Text
- ⑤ 플랜

액티브 세션 락트리 프로세스 정보 병렬 쿼리 트리

17:06:35

	25	28	31	35										
	instance	con_name	sid	serial#	cpu(xos)	pga memory	username	last_call_et	event	query_text	sql_id	session logical...	physical	
	DMX-oracle-demo		52	9934			HR	20	PL/SQL lock timer	declare aa varchar2(10); bb number; cc numbe...	7h2bxycbwptww			
	DMX-oracle-demo		61	4087			HR	17	enq: TX - row lock co...	select /*bind_test*/ from bind_test where regi...	68vqbma8ubrhz			

누가 처음 잡았고, 어디까지 막혔는지 — 한 화면에서.

- 최상위 holder

누가 chain의 root인가 — 원인 세션 식별
- waiter chain

어디까지 막혀 있는가 — 영향 범위
- 락 대기 트렌드 · 데드락 시계열

단발인지 누적인지 — 같은 시간축에서

SQL Elopse Map

어제부터 갑자기 느린 쿼리는 여기서 가장 먼저 잡힌다

인스턴스 모니터링 WhaTap AI

시간

LIVE 18:03:26

10분

인스턴스

DMX-oracle-demo

데이터베이스

orcl

SQL Elopse Map

Query List

* runtime으로 정렬한 상위 200건을 보여줍니다.

instance	time	sid	serial#	con_name	schemaname	elapsed_time (sec)	query
DMX-oracle-demo	2026-05-21 17:57:33	29	16214		HR	20.7	declare aa varchar2(10); bb numbe
DMX-oracle-demo	2026-05-21 17:57:12	60	18375		HR	20.2	declare aa varchar2(10); bb numbe
DMX-oracle-demo	2026-05-21 17:57:40	42	56594		HR	19.2	SELECT /*+ procedure_test1 FULL(
DMX-oracle-demo	2026-05-21 17:57:12	29	16214		HR	17.1	select /*bind_test*/ from bind_tes

인스턴스 모니터링

WhaTap AI

시간



LIVE 18:03:26 10분

인스턴스



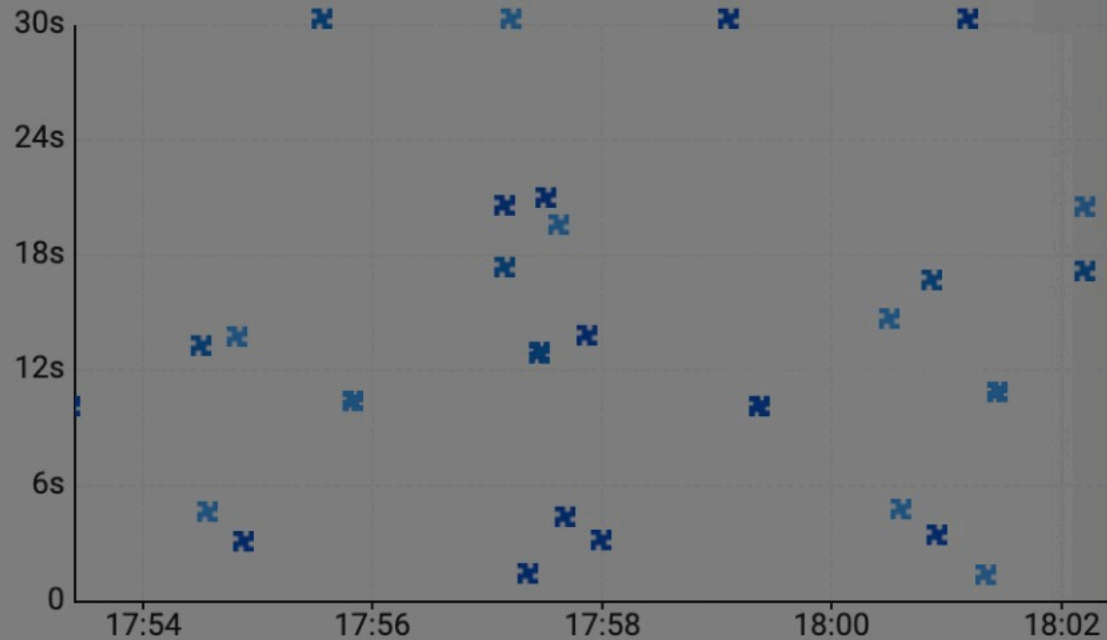
DMX-oracle-demo



데이터베이스

orcl

SQL Elapse Map



1 / 2

SQL Elapse Map — 시간축 × 수행시간

점이 위로 갈수록 느린 쿼리 — 의심 시간대를
드래그

SQL Elapse Map

어제부터 갑자기 느린 쿼리는 어떤 것일까?

2 / 2

그 구간의 SQL 목록 — Query List

elapsed_time 순으로 펼쳐짐 — 행을 클릭하면 실제 SQL Text

인스턴스 모니터링 WhaTap AI

시간 LIVE 18:03:26 10분 인스턴스 DMX-oracle-demo 데이터베이스 orcl

SQL Elapse Map

* runtime으로 정렬한 상위 200건을 보여줍니다.

instance	time	sid	serial#	con_name	schemaname	elapsed_time (sec)	query
DMX-oracle-demo	2026-05-21 17:57:33	29	16214		HR	20.7	declare aa varchar2(10); bb numbe
DMX-oracle-demo	2026-05-21 17:57:12	60	18375		HR	20.2	declare aa varchar2(10); bb num
DMX-oracle-demo	2026-05-21 17:57:40	42	56594		HR	19.2	SELECT /*+ procedure_test1 FU
DMX-oracle-demo	2026-05-21 17:57:12	29	16214		HR	17.1	select /*bind_test*/ from bind_tes

Query List

* runtime으로 정렬한 상위 200건을 보여줍니다.

instance	time	sid	serial#	con_name	schemaname	elapsed_time (sec)	query
DMX-oracle-demo	2026-05-21 17:57:33	29	16214		HR	20.7	declare aa varchar2(10); bb num
DMX-oracle-demo	2026-05-21 17:57:12	60	18375		HR	20.2	declare aa varchar2(10); bb num
DMX-oracle-demo	2026-05-21 17:57:40	42	56594		HR	19.2	SELECT /*+ procedure_test1 FU
DMX-oracle-demo	2026-05-21 17:57:12	29	16214		HR	17.1	select /*bind_test*/ from bind_tes

플랜 분석 — 옵티마이저가 어떻게 푸는지

코드는 안 바꿨는데 어제부터 느린 이유는 보통 여기서

SQL 상세 SQL 통계 보기 →

Query (id: c14ctw9ttuz8)

sid 55 serial# 43251
sql_hash_value 1939663848 plan_hash_value 2229184788
schema_name HR machine oracle-demo
program sqlplus@oracle-demo (TNS V1- module SQL*Plus
V3)
osuser oracle

Formatting

```
select /* bind test */ 'aaa' name, 'bbb' job_title, manager_id sal from
departments where manager_id = :mid
union
select e.FIRST_NAME, j.job_title, sum(salary) from employees e,
DEPARTMENTS d, jobs j, job_history h,
countries c,
locations l
where e.DEPARTMENT_ID in (select d1.DEPARTMENT_ID from
DEPARTMENTS d1, EMPLOYEES e2 where
substr(d1.department_name,1,5) like '%'||substr(e.last_name,1,2)||'%' )
group by e.FIRST_NAME, j.job_title
union
select e.FIRST_NAME, j.job_title, sum(salary) from employees e,
DEPARTMENTS d, jobs j, job_history h
where e.DEPARTMENT_ID in (select d1.DEPARTMENT_ID from
DEPARTMENTS d1, EMPLOYEES e2 where
substr(d1.department_name,1,5) like '%'||substr(e.first_name,1,2)||'%' )
group by e.FIRST_NAME, j.job_title
```

Trend Runtime Plan Plan History Bind Capture Explain Plan

Time 2026/05/21 17:18:35 ~ 2026/05/21 18:18:35 최근 1시간 최근 1일

SQL Statistics

Execution Count	Avg. Elapsed Time	Avg. Logical Reads	Avg. Physical Reads	Avg. Redo Size
32	97	4,063.28	0.5	0

plan_hash_value	Execute Count	Avg. Elapsed Time	Avg. Logical Reads	Avg. Physical Reads	Avg. Redo Size
2229184788	31	97.51	4,071.03	0.45	0
3211808872	1	81.29	3,823	2	0

Table Flow

COST	OPERATION	OBJECT_NAME	CARDINALITY
3,322,545	SELECT STATEMENT		
3,322,545	SORT (UNIQUE)		58
	UNION-ALL		
3	TABLE ACCESS (FULL)	DEPARTMENTS	1
3,314,723	HASH (GROUP BY)		19
	FILTER		
3,301,737	MERGE JOIN (CARTESIAN)		315,623,250
132,544	MERGE JOIN (CARTESIAN)		11,689,750
5,778	MERGE JOIN (CARTESIAN)		467,590
264	MERGE JOIN (CARTESIAN)		20,330
5	MERGE JOIN (CARTESIAN)		190
1	INDEX (FULL SCAN)	HR\$ST_EMPLOYEE_IX	10
4	BUFFER (SORT)		19
	INDEX (FAST FULL SCAN)	IX_JOB_TITLE	19
264	BUFFER (SORT)		107
1	TABLE ACCESS (FULL)	EMPLOYEES	107
5,776	BUFFER (SORT)		23

표준 문서 — 옵티마이저가 어떻게 푸는지

SQL 상세 SQL 통계 보기 →

Query (id: c14ctw9tttuz8)

sid 55

sql_hash_value 1939663848

schema_name HR

program sqlplus@oracle-demo (TNS V1-V3)

osuser oracle

serial# 43251

plan_hash_value 2229184788

machine oracle-demo

module SQL*Plus

☐ Formatting

1 / 3

plan_hash_value 변경

같은 SQL인데 hash가 다름 — 옵티마이저
판단이 바뀐 신호

```
group by e.FIRST_NAME, j.job_title
union
select e.FIRST_NAME, j.job_title, sum(salary) from employees e,
DEPARTMENTS d, jobs j, job_history h
where e.DEPARTMENT_ID in (select d1.DEPARTMENT_ID from
DEPARTMENTS d1, EMPLOYEES e2 where
substr(d1.department_name,1,5) like '%(substr(e.first_name,1,2))%' )
group by e.FIRST_NAME, j.job_title
```

COST	OPERATION	
3,322,545	SELECT STATEMENT	
3,322,545	SORT (UNIQUE)	
	UNION-ALL	
3	TABLE ACCESS (FULL) OF DEPARTMENTS	
3,314,723	HASH (GROUP BY)	19
	FILTER	
3,301,737	MERGE JOIN (CARTESIAN)	315,623,250
132,544	MERGE JOIN (CARTESIAN)	11,589,750
5,778	MERGE JOIN (CARTESIAN)	467,590
264	MERGE JOIN (CARTESIAN)	20,330
5	MERGE JOIN (CARTESIAN)	190
1	INDEX (FULL SCAN)	HR.EMPLOYEE_ID_X
4	BUFFER (SORT)	
	INDEX (FAST FULL SCAN)	HR.JOB_TITLE
264	BUFFER (SORT)	
1	TABLE ACCESS (FULL)	EMPLOYEES
5,776	BUFFER (SORT)	

플랜 분석 — 옵티마이저가 어떻게

코드는 안 바꿨는데 어제부터 느린 이유는 보통 여기서

2 / 3

access path 변화 — TABLE ACCESS (FULL)

index range scan → table full scan 등 경로
변경

OPERATION	OBJECT_NAME
SELECT STATEMENT	
SORT (UNIQUE)	
UNION-ALL	
TABLE ACCESS (FULL)	DEPARTMENTS
HASH (GROUP BY)	
FILTER	
MERGE JOIN (CARTESIAN)	
MERGE JOIN (CARTESIAN)	
MERGE JOIN (CARTESIAN)	
MERGE JOIN (CARTESIAN)	
MERGE JOIN (CARTESIAN)	
INDEX (FULL SCAN)	JHIST_EMPLOYEE_IX
BUFFER (SORT)	
INDEX (FAST FULL SCAN)	IX_JOB_TITLE

1
5,776
TABLE ACCESS (FULL)
EMPLOYEES
107
23
BUFFER (SORT)

플랜 분석

코드는 안 바꿨는데 어째서

옵티마이저가 어떻게 하는지

3 / 3

row estimate 차이 — CARDINALITY

통계와 실제 카디널리티 차이 — plan이 풀스캔으로 떨어지는 원인

plan_hash_value	Execute Count	Avg. Elapsed Time	Avg. Logical Reads	Avg. Physical Reads	Avg. Redo Size
2229184788	31	97.51	4,071.03	0.45	0
3211808872	1	81.29	3,823	2	0

COST	OPERATION	OBJECT_NAME	CARDINALITY
3,322,545	SELECT STATEMENT		
3,322,545	SORT (UNIQUE)		58
	UNION-ALL		
3	TABLE ACCESS (FULL)	DEPARTMENTS	1
3,314,723	HASH (GROUP BY)		19
	FILTER		
3,301,737	MERGE JOIN (CARTESIAN)		315,623,250
132,544	MERGE JOIN (CARTESIAN)		11,689,750
5,778	MERGE JOIN (CARTESIAN)		467,590
254	MERGE JOIN (CARTESIAN)		20,330
5	MERGE JOIN (CARTESIAN)		190
1	INDEX (FULL SCAN)	JHIST_EMPLOYEE_IX	10
4	BUFFER (SORT)		19
	INDEX (FAST FULL SCAN)	IX_JOB_TITLE	19
254	BUFFER (SORT)		107
	BUFFER (SORT)		23

실제 장애는 한 화면으로 끝나지 않는다

액티브 세션 쌓임 + Lock Wait 증가 + Plan 변경이 동시에



①~⑤ 진단 루트는 제품과 무관 — 한 화면이 아니라 흐름으로 봐야 잡힙니다.

서비스가 복잡해지면

외부 연동 · 재현 안 되는 컴플레인 — 두 가지 주제

외부 연동

결제 · 메시징 · PG 의존

외부 호출이 응답시간 점령

사례: 외부 PG 4.8초

재현 불가

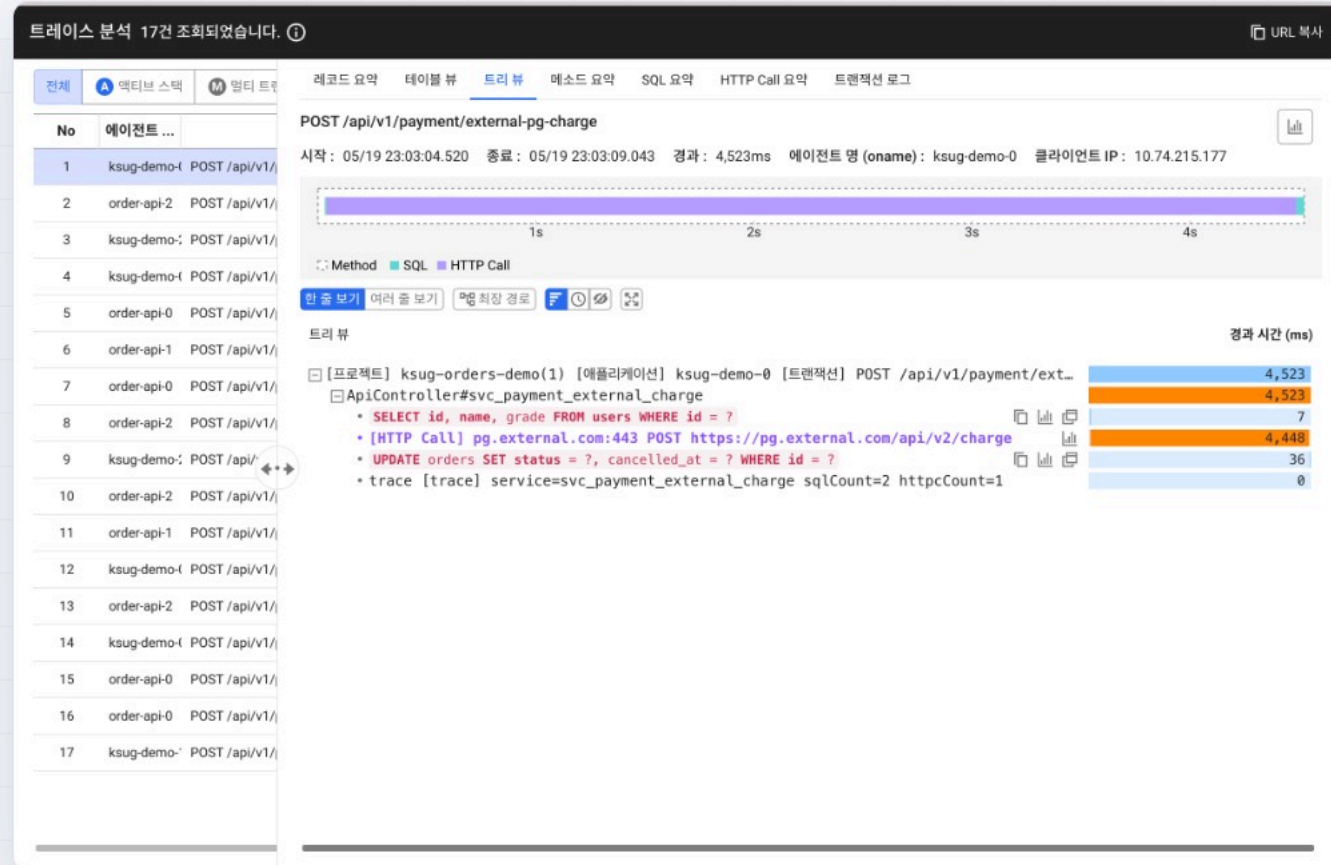
VIP 사용자만 느리다

NPE 났는데 누가·언제 모름

로컬에선 재현 안 됨

외부 시스템의 문제

내부 SQL은 50ms 이내, 외부 PG가 4.4초



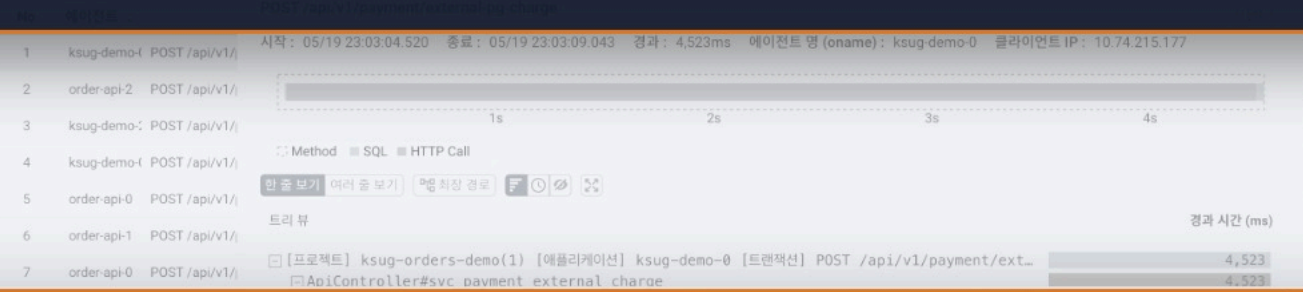
외부 시스템의 문제

내부 SQL은 50ms 이내, 외

1 / 3

외부 PG 4,448ms — 거의 전부가 여기

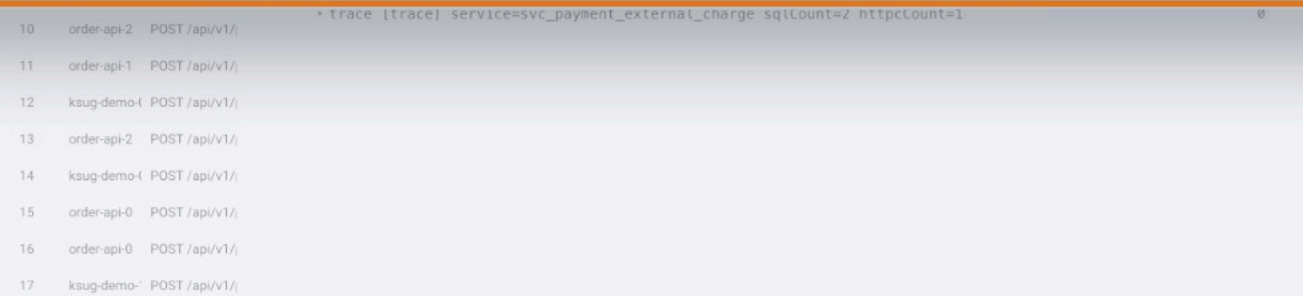
[HTTP Call] pg.external.com/api/v2/charge 한 호출이 전체 시간을 점령



- [HTTP Call] pg.external.com:443 POST https://pg.external.com/api/v2/charge



4,448



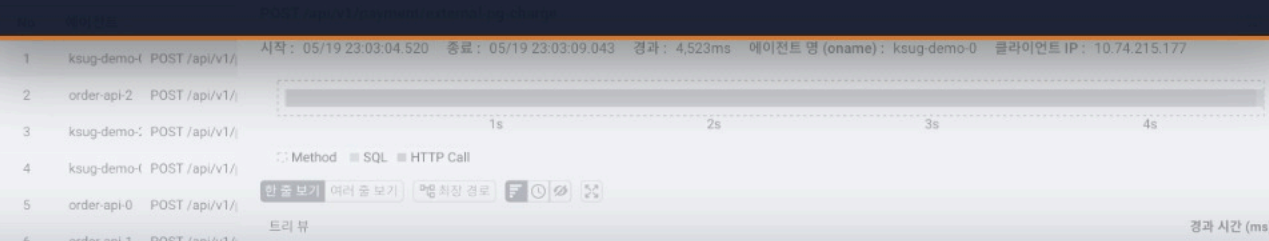
외부 시스템의 문제

내부 SQL은 50ms 이내, 외부 PG?

2 / 3

내부 SQL은 합쳐도 50ms 이내

SELECT 7ms · UPDATE 36ms — 우리 코드/쿼리는 멀쩡



- **SELECT id, name, grade FROM users WHERE id = ?**
- **[HTTP Call] pg.external.com:443 POST https://pg.external.com/api/v2/charge**
- **UPDATE orders SET status = ?, cancelled_at = ? WHERE id = ?**



7,525

7

4,448

36

12 ksug-demo-4 POST /api/v1/

13 order-api-2 POST /api/v1/

14 ksug-demo-4 POST /api/v1/

15 order-api-0 POST /api/v1/

16 order-api-0 POST /api/v1/

17 ksug-demo-4 POST /api/v1/

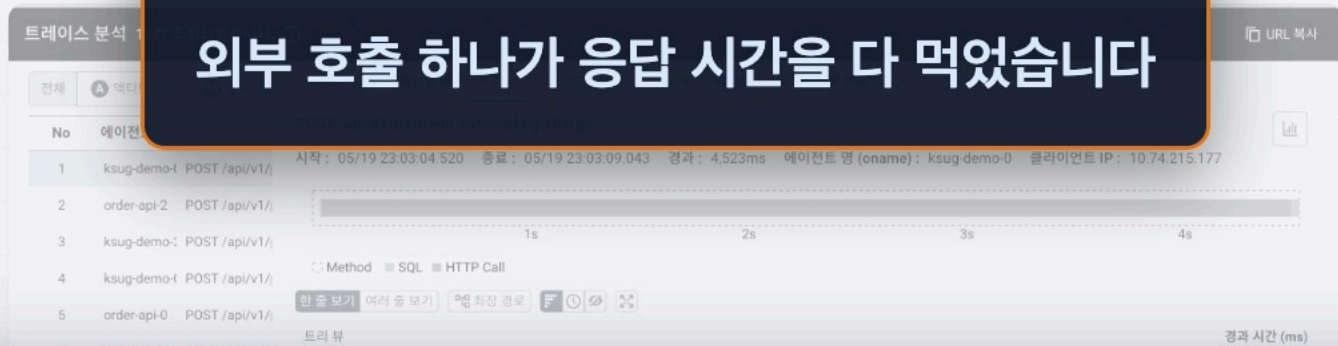
외부 시스템의 문제

내부 SQL은 50ms 이내, 외부 PG가 4초

3 / 3

전체 응답 4,523ms

외부 호출 하나가 응답 시간을 다 먹었습니다



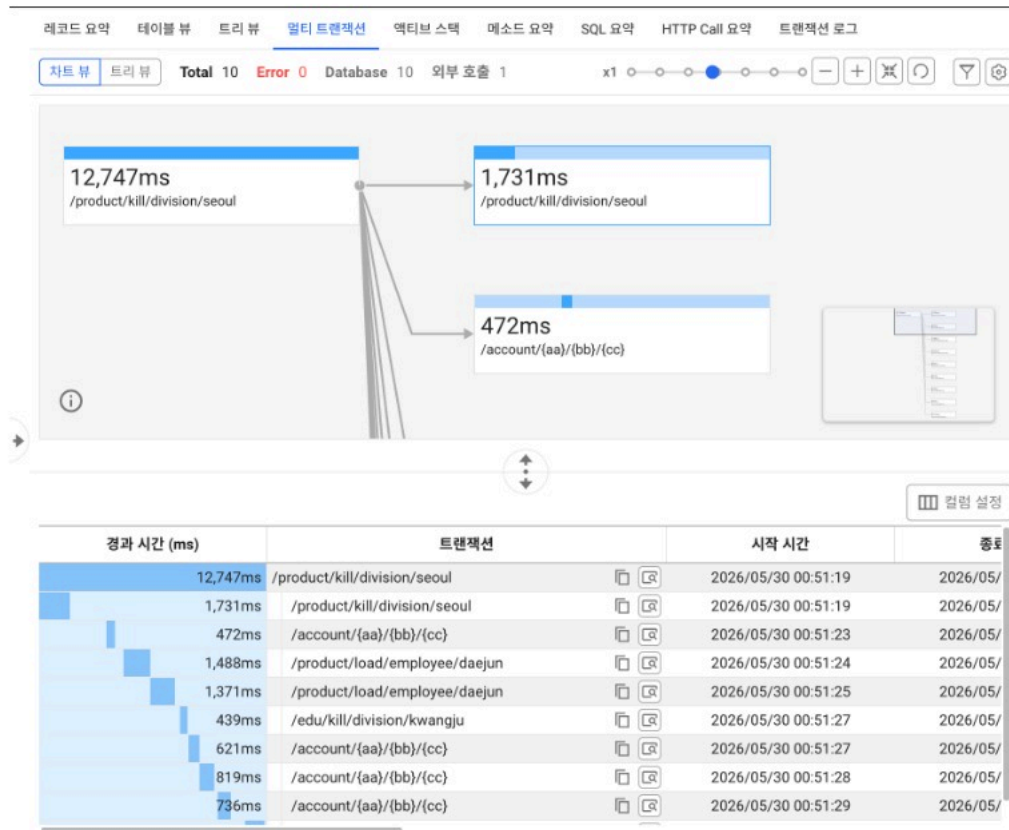
□ [프로젝트] ksug-orders-demo(1) [애플리케이션] ksug-demo-0 [트랜잭션] POST /api/v1/payment/ext...	4,523
□ ApiController#svc_payment_external_charge	4,523
• SELECT id, name, grade FROM users WHERE id = ?	7
• [HTTP Call] pg.external.com:443 POST https://pg.external.com/api/v2/charge	4,448
• UPDATE orders SET status = ?, cancelled_at = ? WHERE id = ?	36
• trace [trace] service=svc_payment_external_charge sqlCount=2 httpcCount=1	0

16 order-api-0 POST /api/v1/

17 ksug-demo-0 POST /api/v1/

MSA 공방을 끝내는 한 장

부모 12.7초, 자식 1.7초 + 0.5초 — 어느 호출이 시간을 먹는지 한 장에



한 장에서 보이는 것

`/product/kill/division/seoul`

12,747ms 부모가 1,731ms + 0.5초 자식을 호출
— 어떤 서비스가 시간을 먹는지 즉시
좁혀집니다.

효과

문제 구간을 한 장으로 좁힌다 — 지연·오류가
같은 타임라인 위에.

로그 - 재현이 안 됩니다

간헐 에러 · 로컬 재현 불가 · 컨텍스트 부재

찍힌 것

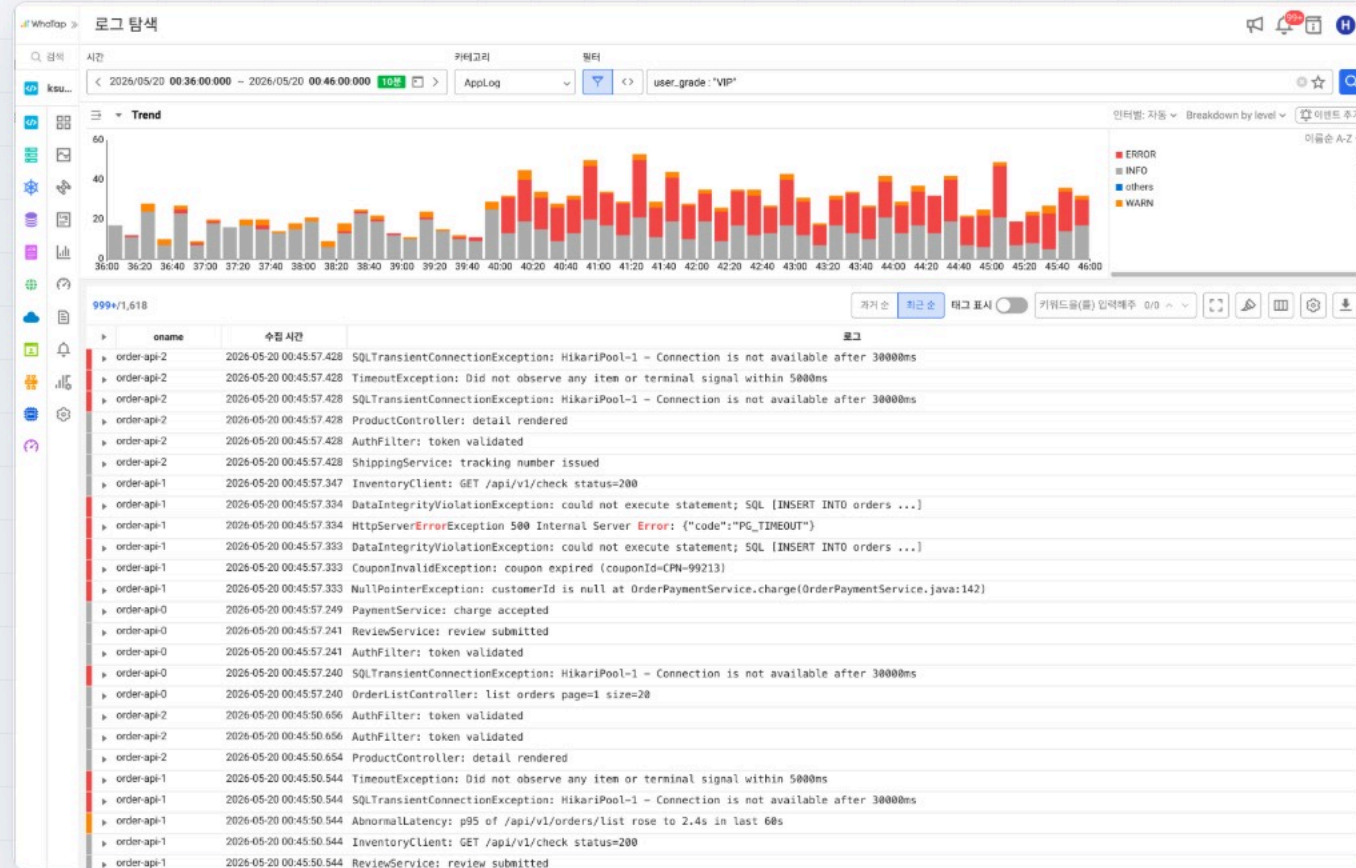
NullPointerException
at OrderService.java:142
스택 트레이스

빠진 것

언제 발생?
누가 요청?
어떤 파라미터?

user_grade: "VIP" 한 줄로 분리

웹 접속 / grep 없이 — Context 한 줄로 즉시 좁히기



필터



<>

user_grade : "VIP"

1 / 3

필터 한 줄로 좁힘

`user\_grade: "VIP"` — 웹 접속·grep 없이
UI에서 즉시

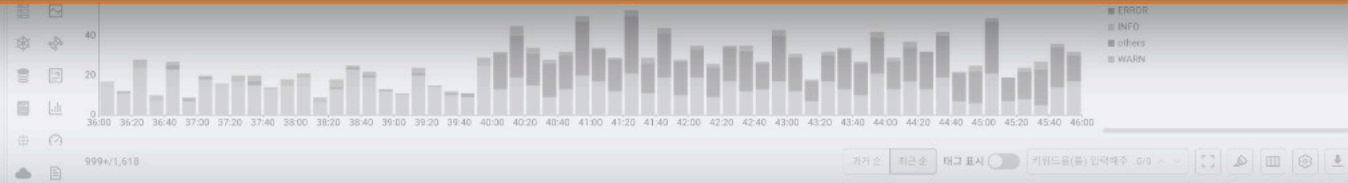
```
» order-api-2 2026-05-20 00:45:57.428 TimeoutException: Did not observe any item or terminal signal within 5000ms
» order-api-2 2026-05-20 00:45:57.428 SQLTransientConnectionException: HikariPool-1 - Connection is not available
» order-api-2 2026-05-20 00:45:57.428 ProductController: detail rendered
» order-api-2 2026-05-20 00:45:57.428 AuthFilter: token validated
» order-api-2 2026-05-20 00:45:57.428 ShippingService: tracking number issued
» order-api-1 2026-05-20 00:45:57.347 InventoryClient: GET /api/v1/check status=200
» order-api-1 2026-05-20 00:45:57.334 DataIntegrityViolationException: could not execute statement; SQL [INSERT INTO orders ...]
» order-api-1 2026-05-20 00:45:57.334 HttpServerErrorException 500 Internal Server Error: {"code":"PG_TIMEOUT"}
» order-api-1 2026-05-20 00:45:57.333 DataIntegrityViolationException: could not execute statement; SQL [INSERT INTO orders ...]
» order-api-1 2026-05-20 00:45:57.333 CouponInvalidException: coupon expired (couponId=CPH-99213)
» order-api-1 2026-05-20 00:45:57.333 NullPointerException: customerId is null at OrderPaymentService.charge(OrderPaymentService.java:142)
» order-api-0 2026-05-20 00:45:57.249 PaymentService: charge accepted
» order-api-0 2026-05-20 00:45:57.241 ReviewService: review submitted
» order-api-0 2026-05-20 00:45:57.241 AuthFilter: token validated
» order-api-0 2026-05-20 00:45:57.240 SQLTransientConnectionException: HikariPool-1 - Connection is not available after 30000ms
» order-api-0 2026-05-20 00:45:57.240 OrderListController: list orders page=1 size=20
» order-api-2 2026-05-20 00:45:50.656 AuthFilter: token validated
» order-api-2 2026-05-20 00:45:50.656 AuthFilter: token validated
» order-api-2 2026-05-20 00:45:50.654 ProductController: detail rendered
» order-api-1 2026-05-20 00:45:50.544 TimeoutException: Did not observe any item or terminal signal within 5000ms
» order-api-1 2026-05-20 00:45:50.544 SQLTransientConnectionException: HikariPool-1 - Connection is not available after 30000ms
» order-api-1 2026-05-20 00:45:50.544 AbnormalLatency: p95 of /api/v1/orders/list rose to 2.4s in last 60s
» order-api-1 2026-05-20 00:45:50.544 InventoryClient: GET /api/v1/check status=200
» order-api-1 2026-05-20 00:45:50.544 ReviewService: review submitted
```

user_grade: "VIP" 한 줄로 분과 2 / 3

웹 접속 / grep 없이

Trend 차트 — 시간대별 ERROR/WARN

"언제 에러가 몰렸는지"가 한 눈에 — 분포가 즉시 보임



시간

카테고리

필터

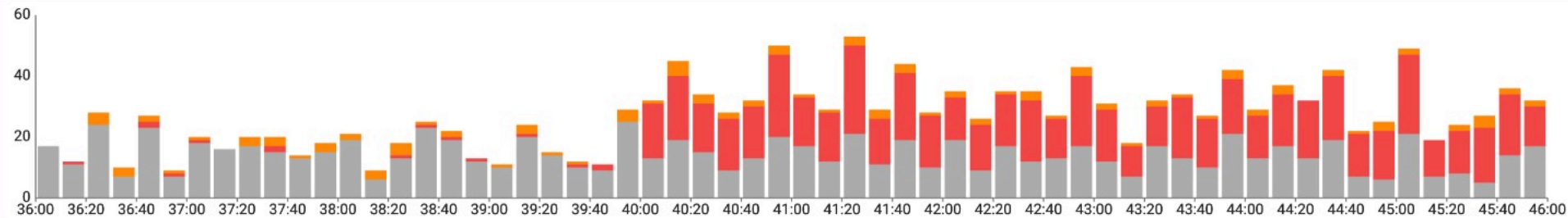
< 2026/05/20 00:36:00:000 ~ 2026/05/20 00:46:00:000 10분 >

AppLog

user_grade : "VIP"

Trend

인터벌: 자동 Breakdown by level



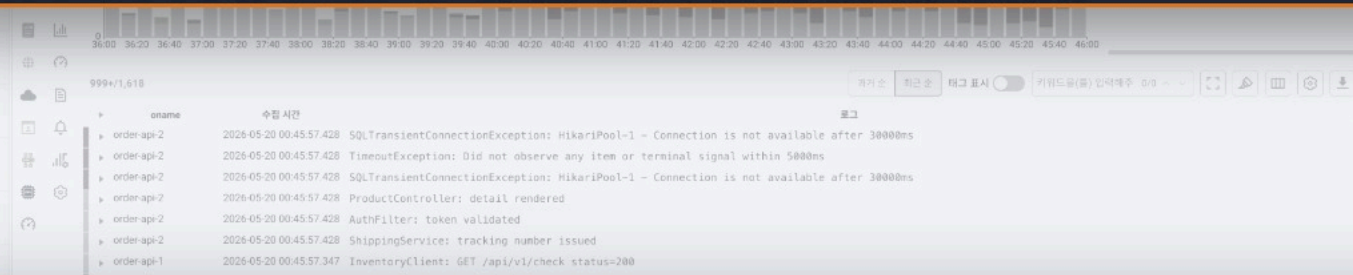
```
2026-05-20 00:45:50.544 AbnormalLatency: p95 of /api/v1/orders/list rose to 2.4s in last 60s
2026-05-20 00:45:50.544 InventoryClient: GET /api/v1/check status=200
2026-05-20 00:45:50.544 ReviewService: review submitted
```

user_grade: "VIP" 한 줄로 분리 3 / 3

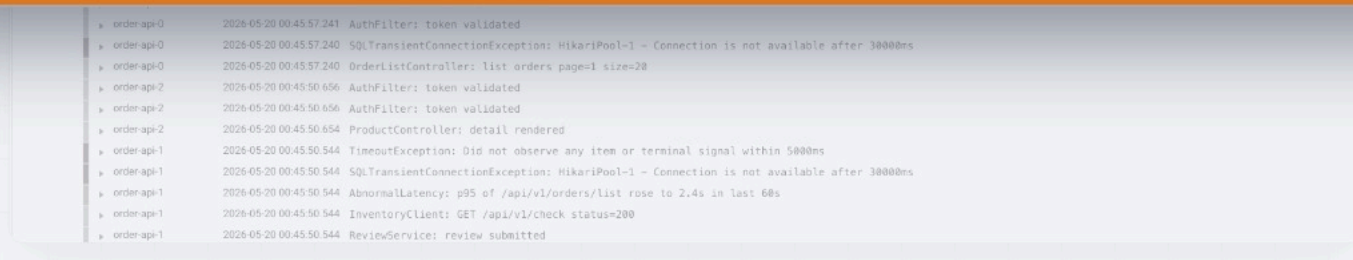
셸 접속 / grep 없이 — Context 한 줄로 찾기

NPE — Context와 같이

NullPointerException at OrderPaymentService.charge(java:142) — Exception만 찍히던 것이 이제 user/time/요청과 함께



2026-05-20 00:45:57.333 CouponInvalidException: coupon expired (couponId=CPN-99213)



같은 에러, 컨텍스트가 같이 찍히면

userId · itemId · itemStatus → 패턴이 보인다

Before

```
ERROR: NullPointerException at OrderService.charge(OrderService.java:142)
```

After

```
ERROR ... userId=u_a9f2x4 itemId=A99 itemStatus=DEPRECATED deployVersion=v2.1.1
```

itemStatus=DEPRECATED + **deployVersion=v2.1.1** 조합에서만 NPE — 며칠 → 몇 분

배포 후 1시간을 넘어서

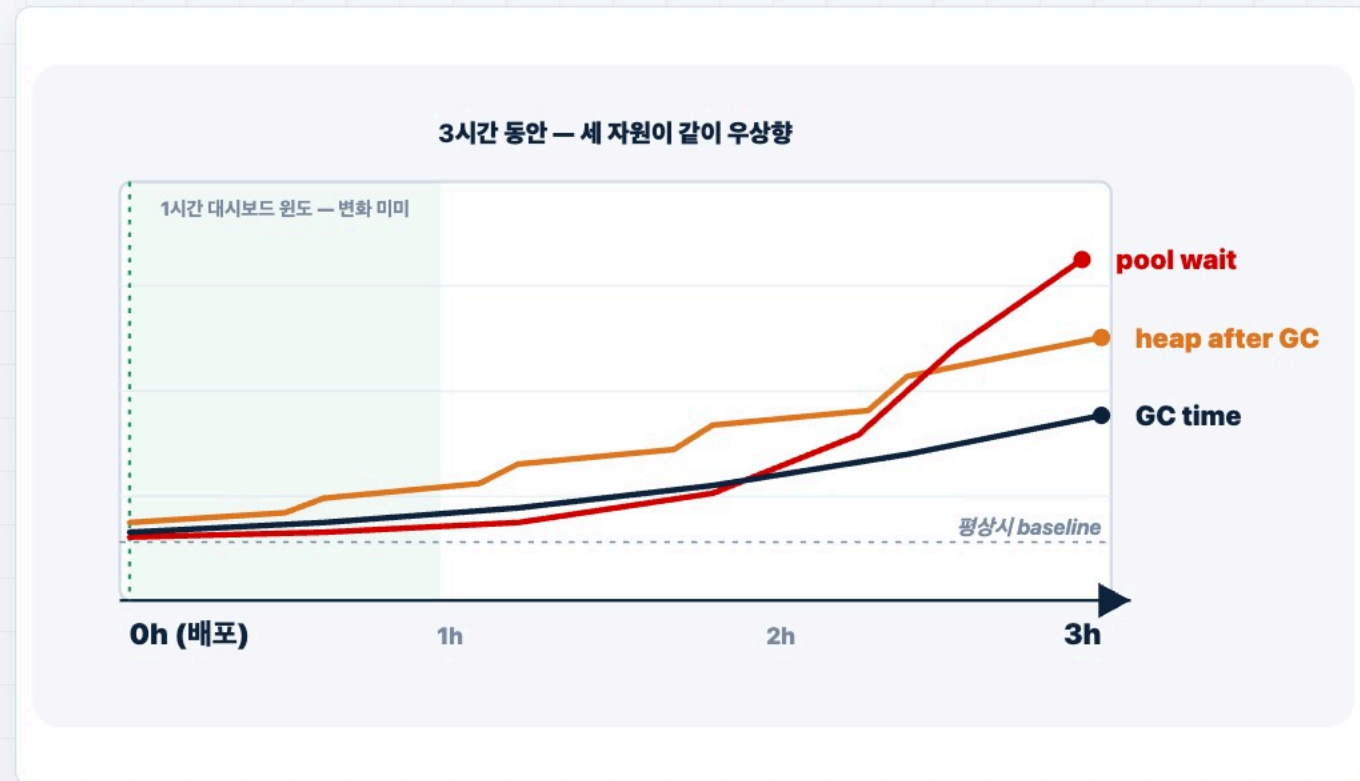
1시간 안에 보이는 문제와, 시간이 지나야 드러나는 문제는 다릅니다.



배포 후 1시간만 보면 부족하다 — 3시간, 다음날 피크까지 관찰해야 한다

배포 후 3시간 — 쌓이면서 드러난다

1시간 윈도우를 넘어가는 자원 추세 — 누수·풀 대기·GC 시간



heap after GC ↑

GC를 돌려도 바닥선이 계속 오름 — 누수 의심

pool wait ↑

커넥션 풀 점유 누적 → 대기 시간 가파른 우상향

GC time ↑

GC 빈도·시간이 함께 늘며 CPU 잠식

1시간 대시보드엔 안 보인다 — 자원 추세를 같이 봐야 한다 · 원인 확정 X, 강한 이상 징후

다음날 — 주기 · 피크 · 데이터

1시간 윈도우 밖 — P95(상위 5%) 응답·세션·느린 SQL이 비로소 드러난다

다음날 피크를 통과하기 전엔 '배포 성공'이 아니다



다음날 피크를 통과하기 전엔 '배포 성공'이 아니다

아침 피크

평균은 정상인데 P95(상위 5%) 응답이 길어진다

야간 배치·크론

데이터 증가 → 실행계획 변화 · 카디널리티 저하 · 느린 SQL

긴 시간 차트 + 로그

배포 기능이 피크를 통과했는지 확인

APM과 DB로 모든 문제가 해결되나요?

API는 정상인데 사용자는 느릴 수 있습니다

APM/DB 정상

서버 150ms

SQL 정상

에러율 낮음

사용자 경험 이상

브라우저 3초

네트워크/CDN

JS·렌더링 지연

서버 150ms ≠ 사용자 체감 3초

네트워크 · CDN · JS 번들 · 렌더링은 APM 바깥



APM 막대는 전체의 5%. 나머지 95%는 브라우저 쪽에 있다.

AJAX 분석 31건 조회되었습니다.

전체 ▼ | 탭 설정

시작 시간 (수집 시...)	전체 경...
2026/05/19 23:31:20	12
2026/05/19 23:31:22	10
2026/05/19 23:31:22	11
2026/05/19 23:31:22	11
2026/05/19 23:31:22	9
2026/05/19 23:31:22	9
2026/05/19 23:31:22	9
2026/05/19 23:31:22	9
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7
2026/05/19 23:31:25	7

```

graph LR
    A["7,284ms  
/yard/api/flush"] --> B["44ms  
/yard/api/flush"]
    B --> C["1ms  
/yard/api/flush"]
    B --> D["0ms  
Async@ UserBehaviorYardApiCreationServi..."]
    C --> E["1ms  
/mqx/q/pcode/q/text"]
  
```

Total 5 Error 0

경과 시간 (ms) 트랜잭션 시작 시간 종료 시간 프로젝트

경과 시간 (ms)	트랜잭션	시작 시간	종료 시간	프로젝트
7,284ms	/yard/api/flush	2026/05/19 23:31:25	2026/05/19 23:31:32	운영 RUM 모니터링 287 218.5
44ms	/yard/api/flush	2026/05/19 23:31:29	2026/05/19 23:31:29	운영 서버APM모니터링 1 218.5
1ms	/yard/api/flush	2026/05/19 23:31:29	2026/05/19 23:31:29	운영 서버APM모니터링 1 52.7%
0ms	Async@ UserBehaviorYardApiCreationServi...	2026/05/19 23:31:29	2026/05/19 23:31:29	운영 서버APM모니터링 1 0.0.0
1ms	/mqx/q/pcode/q/text	2026/05/19 23:31:29	2026/05/19 23:31:29	운영 서버APM모니터링 1 10.2%

탭 설정

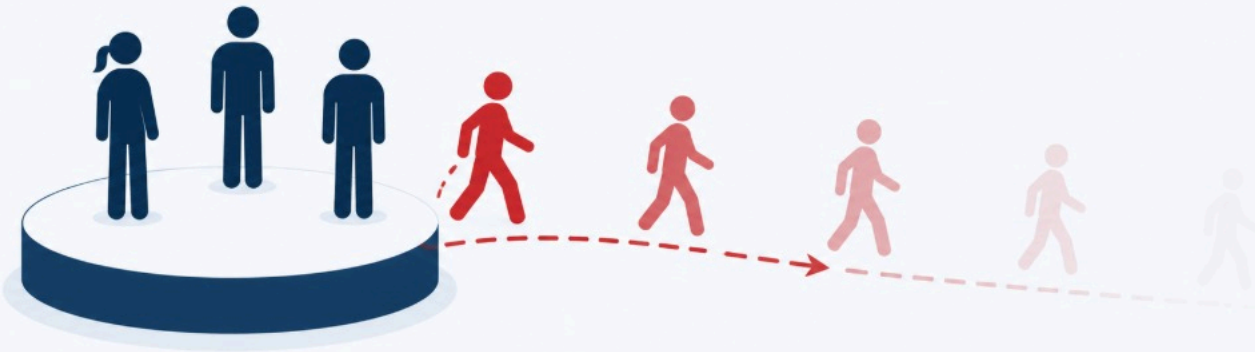
특정 리전/통신사만 느린 문제를 사용자 기준으로 발견

지역 · 통신사 · AJAX 지연 · INP/LCP/CLS

사용자가 무엇을 보았는지를 그대로 재현

사용자 입장에서 보는 게 결국 매출

사용자 경험 저하는 장애가 아니라 매출 손실로 관측된다



서버는 200 OK, 그러나 사용자는 이미 떠나고 있다

10% · 이탈률 증가
1초 지연마다 — BBC

32% · 이탈
한 번의 나쁜 경험만으로 — PwC Future of CX

사례별로 필요했던 제품

처음부터 전부가 아니라, 문제와 성장 단계에 따라 추가



서비스가 커질수록 관측 대상도 서버 → 로그 맥락 → 사용자 경험으로 넓어집니다.

같은 데이터로 책임 공방 없이 동시 대응

개발자 · DBA · 인프라 · PM 이 같은 사건을 같은 화면에서



각자 솔루션이면

다른 도구 · 다른 시간축 · 다른 ID · 다른 해석 →
책임 공방

단일 대시보드면

같은 시간축 · 요청 ID · 배포 시각 · 같은 맥락 →
동시 대응

같은 데이터 위에서 보면, “우리 쪽 아니다”가 아니라 “여기부터 같이 보자”가 된다

AI도 결국 관측의 영역입니다

앞으로 관측 대상은 AI까지 확장된다

AI를 활용한다

AI에게 묻고
답을 이해하고
검증한다

AI를 관측한다

LLM 기능도
새 관측 대상
비용 · 품질 · 지연

AI가 이렇게 말했을 때 이해할 수 있나요?

어휘를 모르면 AI 답도 외계어

 AI DB Connection Pool의 Active Count가 maximumPoolSize에 도달하여 커넥션 획득 대기가 발생하고 있습니다. 평균 대기 시간 24초. HikariCP **maximumPoolSize** 증설 또는 N+1 쿼리 패턴 점검을 권장합니다.

알아야 할 어휘

커넥션 풀

대기시간

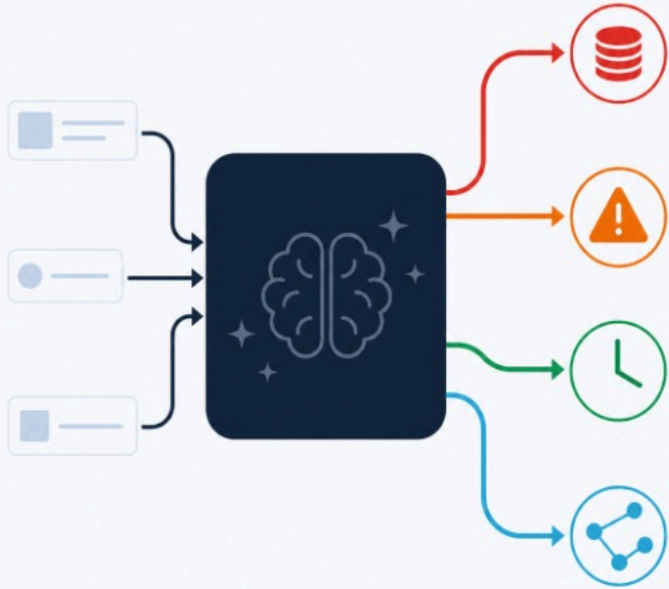
트랜잭션

N+1

오늘 배운 단어가 있어야 AI 분석도 쓸 수 있다.

AI까지 확장되는 관측

질문 맥락 · 비용 · 품질 · 지연만 짧게 본다



질문 맥락

시간대·증상·범위·가설을 함께 준다

비용

요청·기능·모델별 토큰 사용량

품질

환각·근거성·관련성을 확인한다

지연

TTFT·벡터검색·LLM 호출 단계를 나눈다

도구보다 먼저 습관이 남아야 합니다

오늘 가져갈 것 — 화면 이름이 아니라 보는 순서

① 시간 고정

같은 구간으로 비교



② 분포를 본다

평균 말고 P95 · 히트맵



③ 단일 요청으로

트레이스로 내려가기



④ 로그 맥락 연결

누가 · 언제 · 무엇

배포 후 1시간은 시간 고정 → 분포 → 단일 요청 → 로그/트레이스 연결 순서로 본다.

배포 후 1시간, 무엇을 보고 계실 건가요?

코드를 잘 짜는 건 기본 — 다음은, 운영에서 무엇을 보고 판단할지

와탭이든 무엇이든 — 오늘 배포했다면, 지금 트레이스부터 열어보세요.

감사합니다.

